

Comparative Study of Machine Learning Surrogates for Nonlinear Model Predictive Control of Wind Turbine Rotor Speed in Above-Rated Operation

Amidou Maiga, Mustafa Serdar GENÇ, Gamze GENÇ,
Ahmet AKSÖZ, Dongran Song,

Abstract

Wind turbine speed regulation in above-rated operation requires fast, constraint-aware control; nonlinear Model Predictive Control (MPC) is well suited to this, but its internal plant model is expensive to evaluate at every step, motivating cheaper learned surrogates. We compare six surrogate architectures — LLNFM, LSTM, GP, PINN, TCN, and SW-MLP — as internal predictors in a nonlinear MPC for the NREL 5-MW turbine. One-step prediction accuracy and closed-loop tracking prove only loosely coupled. What initially appeared as a hard operational incompatibility between LSTM/TCN/SW-MLP/PINN-v2/GP-v2 and SQP-based `nlmpc` was traced, through a five-stage diagnostic, to repeated `predict()` calls inside the solver — not architecture, model size, or MATLAB version. A verified manual forward pass restored closed-loop viability for all six surrogates, with speedups of $7.8\times$ to $2844\times$. We further identify a Region II/III torque-switching correction as a precondition for stable closed-loop simulation, and a Matérn-kernel GP uncertainty-penalized MPC cost achieving zero pitch activity at matching tracking accuracy. These results reframe an apparent architectural limitation as a diagnosable, remediable implementation cost, with practical guidance for deploying ML surrogates in real-time MPC.

Keywords: Model Predictive Control, Wind Turbine Control, Machine Learning Surrogates, Gaussian Process, Real-Time Feasibility

Contents

1	Introduction	4
1.1	ML-MPC: State of the Art	4
1.2	Contributions	5
1.3	Paper Organization	6
2	System Modeling	7
2.1	NREL 5-MW Reference Turbine	7
2.2	Two-State Model	7
2.3	Region II/III Generator Torque Switching	8
2.4	Wind Model	9
3	Dataset Generation	10
3.1	Simulation Protocol	10
3.2	Dataset Structure	10
3.3	Feature Statistics	10
4	Machine Learning Surrogate Models	12
4.1	V1 Models	12
4.1.1	Local Linear Neuro-Fuzzy Model (LLNFM / ANFIS)	12
4.1.2	Gaussian Process (V1)	12
4.1.3	Physics-Informed Neural Network (V1)	12
4.1.4	Long Short-Term Memory (V1)	13
4.2	V2 Improvements	13
4.2.1	GP-v2 (Matérn 5/2 + Optimized Hyperparameters)	13
4.2.2	PINN-v2 (Differentiable Softplus)	14
4.2.3	TCN and SW-MLP (New Architectures)	14
4.3	Comparative Prediction Accuracy	15
5	MPC Framework	17
5.1	nlimpc Formulation	17
5.2	Surrogate Integration as State Function	17
5.3	GP-MPC Uncertainty Cost (Gap-1)	18
5.4	Operational Feasibility Analysis	19
6	Closed-Loop Results	24
6.1	V1 Results (Baseline, LLNFM, GP, PINN)	24
6.2	V2 Results (Baseline, LLNFM, GP-v2)	25
6.3	V1 vs V2 Comparison	26
6.4	Extended Closed-Loop Evaluation via the <code>predict()</code> Bypass	27

6.5	Extended Monte Carlo Statistical Analysis	31
7	Discussion	35
7.1	Main Findings	35
7.2	Limitations	36
7.3	Perspectives (V3)	38
8	Conclusion	40
A	Manual Forward-Pass Formulas for the predict() Bypass	42
A.1	Feedforward Networks (Residual MLP, SW-MLP, PINN-v2)	42
A.2	Gaussian Process (GP-v2)	42
A.3	Temporal Convolutional Network (TCN)	43
A.4	LSTM	43
A.5	Reproducibility: Explicit Random Seeding	44
A.6	Diagnostic Note on Solver <code>ExitFlag</code>	44

1 Introduction

Wind energy is one of the fastest-growing renewable energy sources, and the continued growth of installed capacity places increasing demands on turbine control systems [Pao and Johnson, 2009]. Above the rated wind speed (Region III), a variable-speed, variable-pitch wind turbine must regulate rotor speed to its rated value by actuating blade pitch, while the generator torque is held at (or switched into) its rated value [Bianchi et al., 2006]. This regulation problem is nonlinear, subject to actuator rate and range constraints, and driven by a highly stochastic disturbance (turbulent wind); Model Predictive Control (MPC) is well-suited to it precisely because it handles input and state constraints explicitly and can anticipate disturbance effects over a prediction horizon [Rawlings et al., 2017], in contrast to classical gain-scheduled PI pitch controllers [Bianchi et al., 2006, Pao and Johnson, 2009, Laks et al., 2009]. The practical obstacle to nonlinear MPC (NMPC) for this problem is computational: at every control step, the solver must repeatedly evaluate the internal plant model — and, for gradient-based solvers, its Jacobian — which is expensive when that model is the full nonlinear aerodynamic and structural dynamics of the turbine [Schlipf et al., 2013, Evans and Lio, 2022].

1.1 ML-MPC: State of the Art

A natural response to this computational bottleneck is to replace the physical plant model inside the MPC prediction step with a learned surrogate that is cheaper to evaluate. Klein et al. [2024, 2025] recently demonstrated this approach for wind turbine load-relevant control using a Local Linear Neuro-Fuzzy Model (LLNFM) surrogate inside an adaptive MPC framework, validated on a full-scale turbine — the direct reference and starting point for the two-state modeling and LLNFM implementation adopted in this work (Section 2). Beyond neuro-fuzzy surrogates, Gaussian Process (GP) regression has an established track record as an MPC-internal model precisely because it returns a calibrated predictive uncertainty alongside its point prediction, enabling cautious or uncertainty-aware control formulations [Kocijan et al., 2004, Hewing et al., 2020] — a property we exploit directly in the GP-based cost of Section 5.3. Within wind energy specifically, Liu et al. [2018] use a Matérn-kernel GP to predict short-term wind speed for preview-based load mitigation, but — unlike the present work — as an external disturbance forecast feeding the controller rather than as the MPC’s internal dynamics model. Data-driven reinforcement learning has separately been applied to wind turbine torque/pitch control [Xie et al., 2023], offering a model-free alternative to the surrogate-based MPC approach studied here; we return to this comparison in Section 7. Physics-Informed Neural Networks (PINNs), which embed a governing differential equation as a soft constraint on the training loss, have become a general framework for combining data-driven flexibility with physical consistency [Raissi et al., 2019, Karniadakis et al., 2021], and Temporal Convolutional Networks (TCNs)

have been shown to be a competitive, often superior alternative to recurrent architectures for generic sequence modeling tasks [Bai et al., 2018]. Each of these four paradigms — neuro-fuzzy, GP, PINN, and convolutional/recurrent sequence models — has therefore been independently validated as a plausible MPC surrogate architecture in some application domain. Beyond ML surrogates specifically, the broader engineering practice of MPC continues to evolve rapidly [Schwenzer et al., 2021], and linear-quadratic-optimal alternatives to nonlinear MPC for wind turbine power tracking, without a learned internal model, have also recently been proposed and validated in OpenFAST [Grapentin et al., 2022, 2024] — a useful point of comparison for the computational-cost trade-offs discussed in Section 5, though outside this paper’s ML-surrogate scope.

What is missing, to our knowledge, is a systematic comparison of these paradigms *within a single closed-loop MPC framework, on the same plant, under the same solver*, paired with an explicit operational feasibility analysis of running each surrogate inside a real-time SQP solve rather than only an open-loop accuracy comparison. Prior work typically evaluates one surrogate family in isolation [Klein et al., 2024, Kocijan et al., 2004, Hewing et al., 2020]; the interaction between surrogate architecture and the specific numerical behavior of a gradient-based NMPC solver — which, as we show in Section 5.4, can render an otherwise-accurate surrogate practically unusable — is, to our knowledge, not addressed in the existing wind turbine ML-MPC literature.

1.2 Contributions

This paper makes four contributions:

1. A systematic comparison of six ML surrogate architectures (LLNFM, LSTM, GP, PINN, TCN, SW-MLP) as internal state predictors for nonlinear MPC of wind turbine rotor speed, evaluated both open-loop (one-step prediction accuracy) and closed-loop (tracking performance under turbulent wind), across two model generations (V1, V2).
2. An operational feasibility analysis documenting a hard incompatibility between `dlarray`-based deep-learning surrogates and SQP-based `nlmpc` in MATLAB R2024a, driven by non-deterministic JIT recompilation rather than a soft accuracy-cost trade-off, with per-call and per-solve timing evidence.
3. A Region II/III generator torque switching correction for the two-state plant model, calibrated at the turbine’s actual rated operating point rather than its theoretical aerodynamic optimum, without which long-horizon closed-loop simulation enters an irrecoverable operating trap — a modeling detail we show is not a minor implementation nicety but a precondition for valid training data and closed-loop evaluation.

4. A Gaussian-Process-based uncertainty-penalized MPC cost, made statistically meaningful through Matérn 5/2 kernel selection and Bayesian hyperparameter optimization, extending the cautious-MPC literature [Hewing et al., 2020, Kocijan et al., 2004] to Region III wind turbine speed regulation.

1.3 Paper Organization

Section 2 presents the plant, torque-switching, and wind models. Section 3 describes the training data generation protocol. Section 4 details all six surrogate architectures and their open-loop accuracy. Section 5 presents the NMPC formulation, surrogate integration, and operational feasibility analysis. Section 6 reports closed-loop V1 and V2 results, their comparison, and a Monte Carlo robustness analysis. Section 7 discusses the main findings, limitations, and directions for future work, and Section 8 concludes.

2 System Modeling

2.1 NREL 5-MW Reference Turbine

This study uses the NREL offshore 5-MW reference wind turbine [Jonkman et al., 2009] as the physical plant. This model is widely adopted as a benchmark in the wind energy control literature, which allows direct comparison with previously published MPC and ML-surrogate studies. The turbine has a rotor radius $R = 63$ m, a combined drivetrain inertia $J = 3.876 \times 10^7$ kg·m² (referred to the low-speed shaft, LSS), a 97:1 gearbox ratio, a rated rotor speed $\omega_{\text{rated}} = 12.1$ rpm, and a rated generator torque (high-speed shaft, HSS) $T_{g,\text{rated}} = P_{\text{rated}}/(\omega_{\text{rated}}N_{\text{gear}}) = 40\,680$ N·m. The rated tip-speed ratio is $\lambda_{\text{rated}} = 7.00$. Full turbine parameters are reported in Table 1.

Table 1: NREL 5-MW reference turbine parameters used in this study (source: `stage0_config.m`).

Parameter	Symbol	Value
Rotor radius	R	63 m
Number of blades	B	3
Air density	ρ	1.225 kg/m ³
Drivetrain inertia (LSS)	J	3.876×10^7 kg·m ²
Drivetrain damping	B_r	6215.5 N·m·s/rad
Gear ratio	N_{gear}	97
Pitch actuator time constant	τ_β	0.1 s
Pitch bounds	β	[0, 25] deg
Maximum pitch rate	$\dot{\beta}_{\text{max}}$	8.0 deg/s
Rated rotor speed	ω_{rated}	12.1 rpm
Physical min. rotor speed	ω_{min}	4.77 rpm
Overspeed limit (+10%)	ω_{max}	13.31 rpm
Rated tip-speed ratio	λ_{rated}	7.00
Rated power	P_{rated}	5 MW
Rated wind speed	V_{rated}	11.4 m/s
Cut-in wind speed	V_{in}	3.0 m/s
Cut-out wind speed	V_{out}	25.0 m/s
Rated generator torque (HSS)	$T_{g,\text{rated}}$	40 680 N·m
Region II gain (LSS)	K_{opt}	2.5106×10^6 N·m·s ²

2.2 Two-State Model

Following the modeling approach of Klein et al. [2024], the turbine is represented by a reduced two-state nonlinear model with state vector $x = [\omega_r, \beta]^\top$, control input $u = \beta_{\text{cmd}}$ (pitch angle demand), and measured disturbance V (rotor-effective wind speed). The rotor dynamics are governed by the aerodynamic torque balance

$$J \dot{\omega}_r = T_a(\omega_r, \beta, V) - T_g^{\text{LSS}}(\omega_r) - B_r \omega_r, \quad (1)$$

where T_g^{LSS} is the generator reaction torque referred to the low-speed shaft (Section 2.3), $B_r = 6215.5 \text{ N m s/rad}$ is a linear drivetrain damping coefficient (Table 1), and the aerodynamic torque is $T_a = \frac{1}{2} \rho \pi R^2 V^3 C_p(\lambda, \beta) / \omega_r$, with tip-speed ratio $\lambda = \omega_r R / V$ and power coefficient surface $C_p(\lambda, \beta)$ evaluated with the Jonkman polynomial. The pitch actuator is modeled as a first-order lag,

$$\tau_\beta \dot{\beta} = \beta_{\text{cmd}} - \beta, \quad (2)$$

with $\tau_\beta = 0.1 \text{ s}$. Equations (1)–(2) are integrated with explicit Euler discretization at $\Delta t = 0.1 \text{ s}$, matching the control sampling time T_s used by the nonlinear MPC in Section 5. The $C_p(\lambda, \beta)$ surface is evaluated strictly within its calibration domain $\beta \in [0 \text{ deg}, 25 \text{ deg}]$; Stage 0 validation confirmed a maximum power coefficient $C_p^{\text{max}} = 0.480$ at $\lambda^* = 8.11$, $\beta^* = 0 \text{ deg}$, safely below the Betz limit ($C_{p,\text{Betz}} = 0.5926$), correcting an earlier non-physical evaluation of the polynomial outside its calibrated range that had produced $C_p^{\text{max}} = 0.706 > C_{p,\text{Betz}}$.

2.3 Region II/III Generator Torque Switching

A methodological contribution of this work concerns the generator torque model $T_g^{\text{LSS}}(\omega_r)$ in Eq. (1). An initial implementation assumed a constant rated torque $T_g^{\text{LSS}} = N_{\text{gear}} T_{g,\text{rated}}$ at all rotor speeds. Under this assumption, long-horizon closed-loop simulation revealed an irrecoverable operating trap: for ω_r below the rated speed, the aerodynamic torque T_a available at low wind speed can fall below the constant demanded generator reaction torque, driving $\dot{\omega}_r < 0$ with no control authority left to arrest the deceleration (the pitch angle is already saturated at its lower bound $\beta = 0 \text{ deg}$), so the rotor stalls toward zero speed.

This was corrected by replacing the constant-torque assumption with the standard Region II/III generator torque switching law, expressed consistently in low-speed-shaft (LSS) torque on both sides,

$$T_g^{\text{LSS}}(\omega_r) = \begin{cases} K_{\text{opt}} \omega_r^2, & \omega_r < \omega_{\text{rated}} \quad (\text{Region II}) \\ N_{\text{gear}} T_{g,\text{rated}}, & \omega_r \geq \omega_{\text{rated}} \quad (\text{Region III}) \end{cases} \quad (3)$$

where $K_{\text{opt}} = \frac{1}{2} \rho A_{\text{rotor}} C_p(\lambda_{\text{rated}}) R^3 / \lambda_{\text{rated}}^3 = 2.5106 \times 10^6 \text{ N.m.s}^2$ is calibrated at the actual Jonkman operating point $\lambda_{\text{rated}} = R \omega_{\text{rated}} / V_{\text{rated}} = 7.00$, rather than the the-

oretical aerodynamic optimum $\lambda^* = 8.11$ identified in Section 2.2. This choice follows standard practice [e.g. Jonkman et al., 2009]: Jonkman’s rated operating point already accounts for drivetrain and electrical losses, so it sits slightly off the pure-aerodynamic C_p peak; calibrating K_{opt} there prioritizes torque continuity at the switching boundary over exact aerodynamic optimality (calibrating instead at $\lambda^* = 8.11$ would produce a 30% discontinuity, versus 2.1% at $\lambda_{\text{rated}} = 7.00$). The resulting continuity ratio is $K_{\text{opt}}\omega_{\text{rated}}^2/(N_{\text{gear}}T_{g,\text{rated}}) = 1.0215$, i.e. a 2.1% torque discontinuity at $\omega_r = \omega_{\text{rated}}$ (equivalently a single-point deficit of 4128 N m, 0.105% of $N_{\text{gear}}T_{g,\text{rated}}$), which is documented and accepted as negligible: the explicit-Euler integration together with continuous turbulent-wind excursions means the simulated system never dwells exactly at $\omega_r = \omega_{\text{rated}}$ to machine precision. Feasibility of Eq. (3) was validated by confirming $T_a(\omega_r, \beta, V) - T_g^{\text{LSS}}(\omega_r) > 0$ for all $\omega_r \in [\omega_{\text{min}}, \omega_{\text{rated}}]$ and all V in the operating envelope, eliminating the trap. Fig. 1 illustrates T_a and $T_g^{\text{LSS}}(\omega_r)$ before and after the correction. This correction also constrains the admissible rotor-speed range used for surrogate-based MPC bounds in Stage 3 (Section 5): the training domain identified in Stage 1 (Section 3) is [8.02, 13.31] rpm, yielding surrogate MPC bounds of [8.52, 12.81] rpm with a 0.5 rpm safety margin on each side, distinct from the wider [11.5, 13.31] rpm defense-in-depth bound retained for the physics-based Baseline controller.

Figure 1: Region II/III generator torque switching: aerodynamic torque T_a and generator torque $T_g(\omega_r)$ before (constant torque) and after (Eq. (3)) correction.

2.4 Wind Model

Turbulent wind input is generated from the Kaimal spectrum defined in IEC 61400-1 [International Electrotechnical Commission, 2005] for Turbulence Class B ($I = 0.16$), with longitudinal turbulence length scale $L_1 = 340.2$ m. Time series are synthesized via inverse FFT of the discretized Kaimal power spectral density. An initial IFFT normalization produced a turbulence intensity that deviated from the target; this was corrected using a deterministic reference-signal scaling approach, matching the achieved standard deviation $\sigma_{\text{actual}} = 2.24$ m/s to the target $\sigma_{\text{target}} = 2.24$ m/s (IEC Class B at the operating mean wind speed), a 0.0% residual error confirmed in Stage 0 validation. The synthesized wind signal is subsequently passed through a second-order Butterworth low-pass filter with cutoff frequency $f_c = 0.3$ Hz, consistent with the rotor-effective wind filtering approach of Klein et al. [2024], before being applied as the disturbance input V to Eq. (1).

3 Dataset Generation

3.1 Simulation Protocol

Training data for the machine learning surrogates is generated by open-loop simulation of the two-state plant model (Section 2.2) under Region II/III torque switching (Section 2.3). A total of 80 trajectories of 300 time steps each are simulated at $\Delta t = 0.1$ s, giving $80 \times 300 = 24,000$ samples. Each trajectory uses a distinct mean wind speed V_{mean} , superimposed with a turbulent Kaimal component (Section 2.4), and a distinct pitch operating point β_{op} ; across the 80 trajectories V_{mean} spans the above-rated envelope up to the cut-out region (e.g. $V_{\text{mean}} = 14.3$ m/s with $\beta_{\text{op}} = 5.8$ deg up to $V_{\text{mean}} = 23.9$ m/s with $\beta_{\text{op}} = 25.0$ deg). The pitch input is excited with a pseudo-random binary sequence (PRBS), $f_{\text{switch}} = 0.5$ Hz, step amplitude $\delta \in [1, 3]$ deg, so that the surrogate training set captures the local input-output sensitivity needed for one-step-ahead prediction. Rotor speed initialization is deliberately widened down into Region II (ω_r as low as ≈ 8 rpm, consistent with the torque-switching-constrained range identified in Section 2.3) so that the resulting surrogates generalize across the Region II/III boundary rather than only in above-rated operation.

3.2 Dataset Structure

Each sample consists of a feature vector $X = [\omega_r, \beta, V, u_{\text{cmd}}]$ and a one-step-ahead target $Y = [\omega_r^+, \beta^+]$, where $u_{\text{cmd}} = \beta_{\text{cmd}}$ is the pitch demand applied at the current step and the superscript $+$ denotes the state at $t + \Delta t$. The 24,000 samples are split at the *trajectory* level, not the sample level, into 56 training, 12 validation, and 12 test trajectories (70%/15%/15%, random seed 42), giving 16,800 training, 3,600 validation, and 3,600 test samples. Splitting by trajectory rather than by individual sample preserves temporal continuity within each subset and prevents information leakage across adjacent time steps of the same trajectory, which is essential for a fair evaluation of the sequence-based surrogates (LSTM, TCN, Section 4) that would otherwise see near-duplicate windows straddling the train/test boundary. Feature and target normalization (z -score) is computed exclusively on the training split and applied identically to validation and test data, avoiding any leakage of validation/test statistics into the training pipeline.

3.3 Feature Statistics

Table 2 summarizes the operating envelope covered by the generated dataset. The rotor speed ranges from 10.28 rpm to 13.31 rpm (the upper bound coinciding with the plant's maximum admissible speed, Section 2.1), the pitch angle spans its full physical domain $[0, 25]$ deg, and the wind speed ranges from 7.85 m/s to the cut-out limit of 25.0 m/s.

The maximum sampled power coefficient is $C_p^{\max} = 0.480$, consistent with the Stage 0 Betz-limit validation (Section 2.2) and confirming that no sample in the dataset violates aerodynamic feasibility. A full dataset validation pass additionally confirmed the absence of NaN/Inf values, physically consistent one-step increments ($\Delta\omega_r$: mean 0.0030 rpm, std 0.0217 rpm, max 0.1273 rpm per step; $\Delta\beta$: mean -0.0006 deg, std 0.3250 deg, max 0.8000 deg per step, consistent with the pitch rate limit $\dot{\beta}_{\max}$), and correctly normalized training features (zero mean, unit variance).

Table 2: Dataset feature statistics (24,000 samples, 80 trajectories).

Feature	Min	Max	Unit
ω_r	10.28	13.31	rpm
β	0.00	25.00	deg
V	7.85	25.00	m/s
u_{cmd}	0.00	25.00	deg
C_p (sample max)	0.480 (Betz = 0.5926)		–

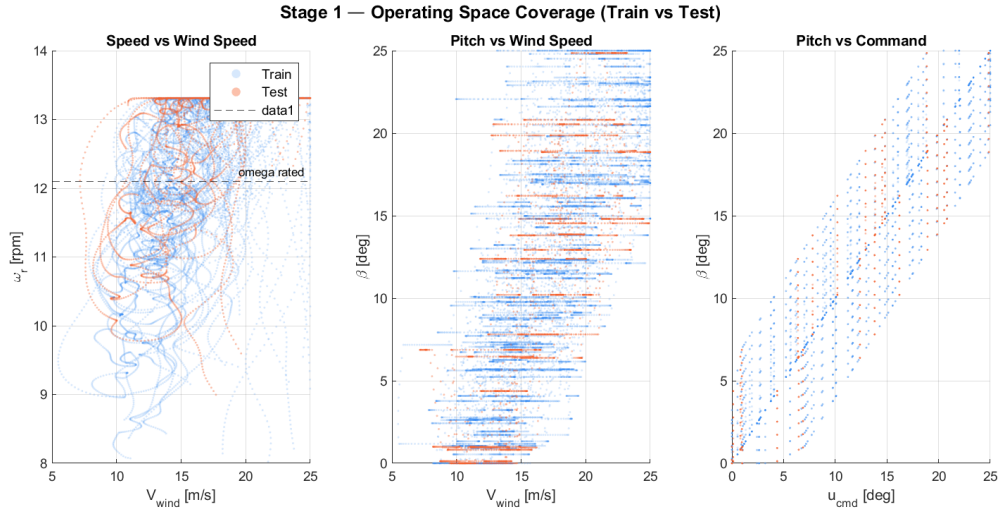


Figure 2: Operating-space coverage of the generated dataset: rotor speed ω_r versus wind speed V , training set (56 trajectories) versus test set (12 trajectories).

4 Machine Learning Surrogate Models

All surrogates are trained on the Stage 1 dataset (Section 3) to predict the one-step-ahead state $[\omega_r^+, \beta^+]$ from $X = [\omega_r, \beta, V, u_{\text{cmd}}]$, using the shared global normalization computed on the training split. Two naive baselines are reported for reference: *Persistence* ($\hat{x}^+ = x$) and a *Linear ARX* model fitted by ordinary least squares. All six surrogate architectures below are trained and evaluated under the same protocol.

4.1 V1 Models

4.1.1 Local Linear Neuro-Fuzzy Model (LLNFM / ANFIS)

The LLNFM surrogate is implemented as two independent Sugeno-type ANFIS networks (one per output channel), each with a grid-partitioned rule base of 2 generalized bell membership functions (`gbellmf`) per input, giving $2^4 = 16$ fuzzy rules, 55 network nodes, and 104 total parameters (80 linear consequent parameters, 24 nonlinear premise parameters). Training uses MATLAB’s `anfis()` function on a 3,000-point subsample of the training set for 80 epochs. We note that the `tunefis()` API (the object-oriented ANFIS training interface introduced in recent MATLAB releases) proved incompatible in R2024a and could not be used; the legacy matrix-based `anfis()` syntax was confirmed to work correctly and was used throughout this study. The omega sub-network converges quickly (error goal reached at epoch 2), while the beta sub-network trains for the full 80 epochs without reaching the error goal, oscillating around a training RMSE of ≈ 0.013 – 0.017 in normalized units. Despite this, LLNFM achieves the best test-set accuracy among all V1 surrogates: $\text{RMSE}_\omega = 0.0069 \text{ rpm}$, $\text{RMSE}_\beta = 0.0936 \text{ deg}$, in 5.8 s of training time.

4.1.2 Gaussian Process (V1)

The V1 Gaussian Process surrogate uses a Squared-Exponential (SE) kernel with default (non-optimized) hyperparameters, fitted independently for each output on a $N_{\text{sub}} = 400$ -point subsample (to keep the $O(N^3)$ training cost tractable). Training completes in 4.0 s, and the model achieves $\text{RMSE}_\omega = 0.0066 \text{ rpm}$, $\text{RMSE}_\beta = 0.1427 \text{ deg}$ on the test set — matching LLNFM as the best-performing V1 surrogate on rotor speed. The mean predictive standard deviation on the test set, $\sigma_{\text{te}} = [0.00108, 0.15139]$ (omega, beta, raw units), is retained for comparison against the calibrated V2 uncertainty (Section 4.2.1).

4.1.3 Physics-Informed Neural Network (V1)

The PINN surrogate is a fully-connected MLP with hidden layers of size 64–64–32, trained for 3,000 iterations with mini-batch size 256 and a step-decayed learning rate (initial 10^{-3} , halved every 400 iterations). The loss combines a data term with a physics residual term derived from the rotor ODE (Eq. (1)), weighted by $\lambda_{\text{phy}} = 0.10$. A physics residual

on the pitch channel was deliberately excluded: since the simulation time step equals the pitch actuator time constant ($\Delta t = \tau_\beta = 0.1\text{ s}$), the exact discrete pitch-actuator ODE degenerates to $\beta^+ = u_{\text{cmd}}$, which conflicts with the pitch-rate saturation present in the real (simulated) data; including this residual was found to actively hurt training. Training converges to a total loss of ≈ 0.0008 by iteration 3,000 (114.5 s total training time), yielding $\text{RMSE}_\omega = 0.0183\text{ rpm}$, $\text{RMSE}_\beta = 0.1924\text{ deg}$ — a 3.1% *degradation* relative to the Linear ARX baseline, the only V1 surrogate to underperform the linear reference.

4.1.4 Long Short-Term Memory (V1)

The LSTM surrogate uses a sequence length of 10 steps and two stacked LSTM layers (64 and 32 units) with 10% dropout, trained for 60 epochs (Adam, learning rate 10^{-3} , batch size 64) on trajectory- respecting sequences (16,240 train / 3,480 validation / 3,480 test sequences, consistent with the trajectory-based split of Section 3.2). Training required 2136.1 s ($\approx 36\text{ min}$) — three orders of magnitude slower than the other V1 surrogates — and achieved $\text{RMSE}_\omega = 0.0147\text{ rpm}$, $\text{RMSE}_\beta = 0.2083\text{ deg}$, a modest +17.1% gain over the linear baseline but markedly worse than LLNFM or GP. As shown in Section 5.4, this accuracy is in any case moot: LSTM inference cost renders the model unusable inside the closed-loop MPC. We attribute the limited benefit of the recurrent architecture to the system’s own dynamics: because $\Delta t = \tau_\beta$, the informative history genuinely required for one-step prediction is short, so a 10-step memory window offers little advantage over the instantaneous state — a hypothesis further supported by the V2 TCN/SW-MLP results (Section 4.2.3).

4.2 V2 Improvements

The V1 dataset (Section 3) is reused unchanged for V2 training. LLNFM is carried over from V1 without modification. Two new sequence-based architectures (TCN, SW-MLP) are introduced to test the memory-window hypothesis raised in Section 4.1.4, and the GP and PINN surrogates are upgraded as described below.

4.2.1 GP-v2 (Matérn 5/2 + Optimized Hyperparameters)

Two changes are introduced relative to the V1 GP. First, the kernel is changed from Squared-Exponential to Matérn 5/2, $k(r) = \left(1 + \frac{\sqrt{5}r}{l} + \frac{5r^2}{3l^2}\right) \exp\left(-\frac{\sqrt{5}r}{l}\right)$, which assumes only twice differentiable sample paths rather than infinite smoothness — a weaker and more physically appropriate regularity assumption for a system subject to turbulent forcing [Rasmussen and Williams, 2006]. Second, hyperparameters (σ_f, l, σ_n) are tuned by Bayesian optimization (marginal likelihood objective, 20 evaluations) rather than left at default values. The optimized hyperparameters differ substantially between the two output channels: for ω_r^+ , $\sigma_f = 0.0013$, $l = 0.0811$, $\sigma_n = 0.0023$; for β^+ , $\sigma_f = 0.4397$,

$l = 24508.4$, $\sigma_n = 0.0729$ — the very large length scale on the beta channel reflecting the near-linear, low-curvature relationship between normalized inputs and β^+ . Training takes 77.9 s (versus 4.0 s for V1, a one-time cost with no impact on inference), and test accuracy improves to $\text{RMSE}_\omega = 0.0102 \text{ rpm}$, $\text{RMSE}_\beta = 0.0692 \text{ deg}$ — a 44.6% gain over the linear baseline and, notably, the best β accuracy of any surrogate evaluated. Critically, the Bayesian-optimized hyperparameters directly calibrate the predictive uncertainty σ_{te} , which motivates the GP-based uncertainty-penalized MPC cost introduced in Section 5.3.

4.2.2 PINN-v2 (Differentiable Softplus)

The V1 PINN clips the power coefficient with $C_{p,\text{clip}} = \max(0, \min(C_{p,\text{Betz}}, C_p))$ to enforce physical bounds in the residual term, which is non-differentiable at the clipping boundaries and produces a zero gradient exactly where the physics loss is most active. In PINN-v2, this is replaced with a smooth softplus saturation, keeping the physics residual differentiable everywhere in the training domain. Training settings are otherwise unchanged (3,000 iterations, $\lambda_{\text{phy}} = 0.10$). Test accuracy is essentially unchanged relative to V1 ($\text{RMSE}_\omega = 0.0193 \text{ rpm}$ vs. 0.0183 rpm , $\Delta \approx 0.0000$ once rounded; $\text{RMSE}_\beta = 0.1636 \text{ deg}$ vs. 0.1924 deg), and PINN-v2 remains, along with TCN, one of the two surrogates that fail to outperform the linear baseline (−4.5%). The differentiable formulation therefore did not translate into a measurable accuracy gain on this dataset, though it removes a documented source of ill-conditioned gradients during training.

4.2.3 TCN and SW-MLP (New Architectures)

Two new sequence-based surrogates are introduced specifically to test whether the poor cost-effectiveness of LSTM (Section 4.1.4) is a limitation of the recurrent architecture itself or a more fundamental consequence of the system’s short effective memory. The *Temporal Convolutional Network* (TCN) uses a 10-step input window with dilated causal 1-D convolutions (dilations 1, 2, 4), 16 filters, kernel size 3, and global average pooling (1,906 parameters total), trained for 60 epochs. The *Sliding-Window MLP* (SW-MLP) instead flattens the same 10-step window into a single 40-dimensional input vector (`featureInputLayer`) fed to a 64–32 MLP (4,770 parameters), trained under the same protocol. Results are markedly different: TCN performs *worse* than persistence ($\text{RMSE}_\omega = 0.0469 \text{ rpm}$, −154.1% vs. linear), while SW-MLP achieves a modest positive gain ($\text{RMSE}_\omega = 0.0171 \text{ rpm}$, +7.5%). Neither architecture approaches the accuracy of the instantaneous-state surrogates (LLNFM, GP-v2). This result reinforces the hypothesis raised in Section 4.1.4: because $\Delta t = \tau_\beta$, the 10-step temporal window is largely redundant information for one-step prediction, and windowed/recurrent architectures pay a real optimization and capacity cost for modeling a temporal dependency the plant does not strongly exhibit at this sampling rate. TCN’s particularly poor result — worse than

naive persistence — suggests the causal-dilated convolutional inductive bias is actively mismatched to this near-memoryless dynamics, though we do not rule out that further hyperparameter tuning could narrow this gap; this is noted as a limitation in Section 7.2.

4.3 Comparative Prediction Accuracy

Table 3 summarizes test-set accuracy for all V1 and V2 surrogates. GP-v2 achieves the best overall accuracy ($\text{RMSE}_\omega = 0.0102 \text{ rpm}$, $\text{RMSE}_\beta = 0.0692 \text{ deg}$), narrowly ahead of V1 GP and LLNFM; LLNFM remains the best accuracy-per-training-second trade-off (5.8s vs. 77.9s for GP-v2). Three surrogates (LSTM, TCN, PINN/PINN-v2) fail to clearly outperform the Linear ARX baseline, and TCN performs worse than even the naive Persistence baseline on this dataset.

We note one clarification for rigor: the V1-to-V2 comparison for *unchanged* components shows a shift that could initially appear to be an unexplained evaluation artifact — e.g. the Persistence baseline moves from $\text{RMSE}_\omega = 0.0214 \text{ rpm}$ (V1) to 0.0241 rpm (V2), and LLNFM, reused without retraining, is reported at $\text{RMSE}_\omega = 0.0069 \text{ rpm}$ in the V1 run versus 0.0106 rpm in the V2 run. Tracing this to its source in `stage1_generate_data.m` confirms a deliberate, documented protocol change rather than an unresolved artifact: the V2 dataset generator widens the initial rotor-speed sampling to explicitly cover Region II ($\approx 8\text{--}12.1 \text{ rpm}$ for odd-indexed trajectories, alongside the V1-style near-rated initialization for even-indexed trajectories, both under fixed, trajectory-indexed random seeds), so that surrogates can learn the Region II/III torque-switching behavior of Section 2.3. Because Persistence involves no learned parameters and LLNFM is evaluated without retraining, both report a genuinely different test-set distribution between V1 and V2 — a wider, harder operating envelope in V2 — rather than a stochastic evaluation difference; the RMSE shift is the expected consequence of that widened envelope, not a discrepancy to be reconciled. V1 and V2 numbers are nonetheless still reported as two separate snapshots, since they were generated by different dataset configurations and are not directly comparable on an absolute basis. Gains relative to Linear ARX (Table 3, last column) are computed within each snapshot for this reason. All V2 figures in Table 3 were re-generated with an explicit, recorded random seed (`cfg.data.split_seed`) fixed before every stochastic training step, confirmed reproducible across repeated runs on the same MATLAB release.

Table 3: Comparative one-step prediction accuracy of all surrogates, V1 and V2 snapshots (test set, $N = 3,600$ samples). V2 figures use explicit, recorded random seeds (Appendix A) and are confirmed reproducible.

Snapshot	Model	RMSE- ω (rpm)	RMSE- β (deg)	Train (s)	Gain vs. Linear
V1	Persistence	0.0214	0.3220	–	–20.6%
	Linear ARX	0.0178	0.1423	< 1	reference
	LLNFM	0.0069	0.0936	5.8	+61.2%
	LSTM	0.0147	0.2083	2136.1	+17.1%
	GP	0.0066	0.1427	4.0	+62.6%
	PINN	0.0183	0.1924	114.5	–3.1%
V2	Persistence	0.0241	0.3220	–	–30.7%
	Linear ARX	0.0184	0.1423	< 1	reference
	LLNFM*	0.0106	0.0971	5.2	+42.3%
	TCN	0.0412	0.2969	432.8	–123.3%
	SW-MLP	0.0159	0.1629	69.1	+14.0%
	GP-v2	0.0102	0.0696	44.3	+44.6%
	PINN-v2	0.0186	0.1846	21.7	–0.8%

*LLNFM architecture and weights unchanged from V1; the RMSE difference vs. V1 reflects evaluation on the wider, Region-II-inclusive V2 test set (see text), not a change to the model itself.

Figure 3: Comparative test-set RMSE across all surrogate models, V1 and V2.

5 MPC Framework

5.1 nlmvc Formulation

Closed-loop control is implemented with MATLAB’s nonlinear MPC object (`nlmpc`), instantiated separately for each surrogate with state vector $x = [\omega_r, \beta]$, output vector $y = x$ (identity output map), manipulated variable $u = \beta_{\text{cmd}}$, and measured disturbance V . All controllers share the sampling time $T_s = 0.1\text{ s}$, prediction horizon $N_p = 10$, and control horizon $N_c = 4$. The standard quadratic cost (used by the Baseline, LLNFM, and PINN controllers) is

$$J = \sum_{k=1}^{N_p} \left[Q (\omega_r(k) - \omega_{\text{rated}})^2 + Q_2 \beta(k)^2 + R (\Delta u(k))^2 \right], \quad (4)$$

with tracking weight $Q = 100$, a small pitch-regulation weight $Q_2 = 0.01$ (encoded via `nlobj.Weights.OutputVariables`), and control-rate weight $R = 0.5$. Input constraints enforce the physical pitch range $\beta \in [0^\circ, 25^\circ]$ and the pitch rate limit $|\Delta\beta| \leq \dot{\beta}_{\text{max}} T_s = 0.8^\circ$ per step (from $\dot{\beta}_{\text{max}} = 8.0^\circ/\text{s}$, Table 1). The rotor-speed output constraint $\omega_r \in [\omega_{\text{min}}^{\text{bound}}, \omega_{\text{max}}^{\text{bound}}]$ is controller-dependent, as detailed in Section 5.2. The SQP solver is configured with bounded worst-case iteration cost (`MaxIterations`= 30, `MaxFunctionEvaluations`= 300, constraint/optimality/step tolerances of 10^{-4}); this deviates from `nlmpc`’s default `MaxIterations`= 400, which was found in early testing to allow the SQP solver to run for over 20 seconds per control step when constraints were difficult to satisfy with an ML surrogate in the loop (observed: 20 624 ms/step for LLNFM-MPC at simulation step $k = 480$). Because each SQP iteration evaluates the surrogate state function multiple times for cost and constraint gradients, this cost scales with surrogate inference latency — a recurring theme in the feasibility analysis of Section 5.4. A real-time controller must have a bounded worst-case computation time [Richter et al., 2012, Xi et al., 2013], so a feasible-but-possibly-suboptimal solution within budget is preferred over an unbounded search for the optimum.

5.2 Surrogate Integration as State Function

Each surrogate is wrapped as an `nlmpc StateFcn` with signature $x^+ = f(x, u, p_1, \dots, p_n)$, where the parameter count `NumberOfParameters` must match between the `StateFcn` and any `CustomCostFcn` (confirmed empirically in R2024a: unused trailing parameters are silently ignored rather than raising an error). The controller signatures used are: Baseline `sf_baseline(x,u,V,p)` ($n = 2$, the true physical plant Eqs. (1)–(2), no surrogate involved); LLNFM and PINN `sf_*(x,u,V,mdl)` ($n = 2$); LSTM `sf_lstm(x,u,V,mdl,seq_buf)` ($n = 3$, carrying an explicit sequence buffer); and GP `sf_gp(x,u,V,mdl,omega_ref,kappa)`

($n = 4$, where `omega_ref` and `kappa` are unused by the state function itself and passed only to satisfy the shared-parameter-count requirement with `cost_gp`, Section 5.3).

The rotor-speed output constraint uses two distinct bounds for two distinct failure modes. The Baseline controller, which evaluates the true plant directly, uses a physics-based defense-in-depth floor $\omega_r \in [11.5 \text{ rpm}, \omega_{\max}]$ ($\omega_{\max} = 13.31 \text{ rpm}$); this bound is not load-bearing for feasibility once the Region II/III torque switching correction (Section 2.3) is in place, but is retained as a cheap safeguard. The four surrogate controllers, in contrast, are *required* to be bounded within the Stage 1 training domain with a 0.5 rpm safety margin, $\omega_r \in [8.52 \text{ rpm}, 12.81 \text{ rpm}]$: this constraint is not optional, since surrogate predictions outside their training domain are unconstrained extrapolations. This was discovered empirically — an early LLNFM-MPC run without this bound drove ω_r down to 7.55 rpm (outside the then-current training range) while the pitch angle saturated at 25 deg, consistent with the surrogate losing sensitivity to the true state once its input left the domain over which its rule base (or, more generally, its learned response surface) was calibrated; a symptom-level input-clamping fix at the surrogate’s input boundary was found to mask rather than resolve the underlying cause, and the correct fix is instead to constrain the MPC’s own admissible operating region.

5.3 GP-MPC Uncertainty Cost (Gap-1)

For the GP controller, the standard quadratic cost is replaced entirely (`Optimization.ReplaceStandard = true`) with a custom cost function that adds a predictive-uncertainty penalty over the prediction horizon:

$$J_{\text{GP}} = \sum_{k=1}^{N_p} \left[Q (\omega_r(k) - \omega_{\text{rated}})^2 + Q_2 \beta(k)^2 + R (\Delta u(k))^2 + \kappa \sigma_{\omega}(k)^2 \right], \quad (5)$$

where $\sigma_{\omega}(k)$ is the GP’s predictive standard deviation on the rotor-speed channel at the predicted state/input pair k steps ahead (re-queried from the GP model at every stage of the prediction horizon, using the same input feature normalization as the state function), and $\kappa = 0.8$ is the uncertainty penalty weight. Because GP-v2’s hyperparameters are calibrated by marginal-likelihood optimization (Section 4.2.1) rather than left at default values, the resulting σ_{ω} is a statistically meaningful (Bayesian) uncertainty estimate rather than an arbitrary scale, which is the methodological justification for using it directly as an MPC cost penalty: the controller is driven to prefer trajectories that stay within regions of the state space where the surrogate is confident, in addition to tracking performance. We note as an implementation detail that the `nlmpc` internal `data` struct in R2024a does not expose a `data.Weights` field (verified by inspecting `fieldnames(data)`, which lists `Ts`, `CurrentStates`, `LastMV`, `References`,

PredictionHorizon, but no Weights); consequently Q , Q_2 , and R are hardcoded inside `cost_gp.m` to match `stage0_config.m`, with no automatic propagation if the configuration changes — a documented limitation of the custom-cost approach in this MATLAB version.

5.4 Operational Feasibility Analysis

This section reports a five-stage diagnostic investigation into why several ML surrogates fail catastrophically inside the closed-loop SQP solve despite being fast in isolation, and how this failure was ultimately resolved. We report the investigation in the order it was conducted, including negative results, because the sequence of ruled-out hypotheses is itself informative for practitioners facing a similar symptom.

Stage 1 — Isolated inference latency is not predictive of closed-loop cost.

An isolated benchmark calling each surrogate’s state function directly (bypassing `nlmpc` entirely, with randomized inputs drawn from the training domain at every call, replicating how SQP probes different state/input pairs) found per-call latencies in the sub-millisecond to low-millisecond range for TCN, SW-MLP, and PINN-v2, with only rare spikes (0–4 calls out of 250 exceeding 50 ms) even for LSTM. Table 4 reports the isolated results.

Table 4: Isolated state-function call latency (250 calls, randomized inputs), MATLAB R2026a.

Surrogate	Median (ms)	Max (ms)	Spikes (>50 ms)
LLNFM	0.25	6.00	0/250
GP (V1)	0.22	21.36	0/250
PINN	0.64	35.88	0/250
TCN	0.65	147.57	2/250
SW-MLP	0.64	3.47	0/250
GP-v2	0.31	5.84	0/250
PINN-v2	0.64	2.75	0/250
LSTM	6.94	321.82	4/250

Stage 2 — Real closed-loop simulation reveals a catastrophe the isolated benchmark did not predict. Running each surrogate inside an actual `nlmpcmove` closed loop (rather than calling the state function directly) produced per-step costs one to four orders of magnitude higher than the isolated benchmark, for every surrogate except LLNFM. TCN, previously the cleanest sequential architecture in isolation, degraded to a mean of 1173.5 ms/step (max 7688.2 ms); SW-MLP, PINN-v2, and GP-v2 each independently degraded to comparable means (1.1–2.3 s/step); LSTM required ≈ 318 s/step. Every one of these controllers exceeded the $T_s = 0.1$ s real-time budget on effectively 100%

of simulation steps. This result is central to our diagnostic: the common factor across all failing architectures is not network size, sequence length, or architecture family, but the presence of a `predict()` call — whether on a `dlnetwork` object (TCN, SW-MLP, PINN-v2, LSTM) or a `RegressionGP` object (GP-v2) — evaluated repeatedly inside `fmincon`’s SQP iterations.

Stage 3 — Supplying an analytical Jacobian does not resolve the catastrophe.

A natural hypothesis is that `fmincon` without a supplied `Jacobian.StateFcn` falls back to global numerical differentiation, re-evaluating the *entire* predicted trajectory once per perturbed decision variable, and that this combinatorial cost — not `predict()` per se — is the true bottleneck. We tested this directly for GP-v2 by supplying a locally-computed (central finite-difference) Jacobian. Performance *did not improve* (mean 2713.5ms/step versus 1233.1ms/step without the supplied Jacobian) and, critically, diagnostic logging revealed `ExitFlag` ≤ 0 (the `MaxIterations`= 30 cap reached without convergence) at *every* simulation step, for *every* controller including the fast, well-behaved Baseline. This ruled out both the numerical-Jacobian hypothesis specifically and, more importantly, revealed that `ExitFlag` alone is not a useful diagnostic of surrogate-specific failure: it reflects a deliberately tight iteration budget (Section 5.1) common to all controllers, not a symptom isolated to ML surrogates.

Stage 4 — Bypassing `predict()` with a hand-written forward pass resolves the catastrophe.

For each surrogate, we extracted its trained weights into plain double-precision arrays and implemented the corresponding forward pass by direct matrix arithmetic — `tanh`/`ReLU` dense layers for the residual MLP, SW-MLP, and PINN-v2; the Matérn 5/2 posterior-mean formula $m(\mathbf{x}) = \beta_0 + \sum_i \alpha_i k(\mathbf{x}, \mathbf{x}_i)$ for GP-v2, using its trained active set, dual coefficients α_i , and constant basis coefficient β_0 ; and, for TCN, the three dilated causal convolution blocks plus per-timestep channel-wise layer normalization — *never* calling `predict()` or constructing a `dlarray`/`RegressionGP` query at simulation time. Each manual implementation was verified against `predict()` on multiple random test points before use (agreement to 10^{-13} – 10^{-7} for the feedforward architectures and GP; see Stage 5 for TCN and LSTM). Table 5 reports the closed-loop result.

Table 5: Closed-loop per-step cost, `predict()` versus manual forward pass, all six surrogates, identical trained weights (verified numerically equivalent to 10^{-13} – 10^{-4} per Section 5.4).

Surrogate	<code>predict()</code> (ms)	Manual (ms)	Speedup	Overruns (manual)
MLP-residual ¹	2188.3	22.3	98×	2/600
GP-v2 ($N_{\text{sub}} = 50$)	196.2	25.2	7.8×	0/600
SW-MLP	1383.9	30.4	45×	1/600
PINN-v2	1650.2	12.1	136×	0/600
TCN (vectorized)	834.1	23.9	35×	8/600
LSTM	148,475.7	52.2	2844×	1/600

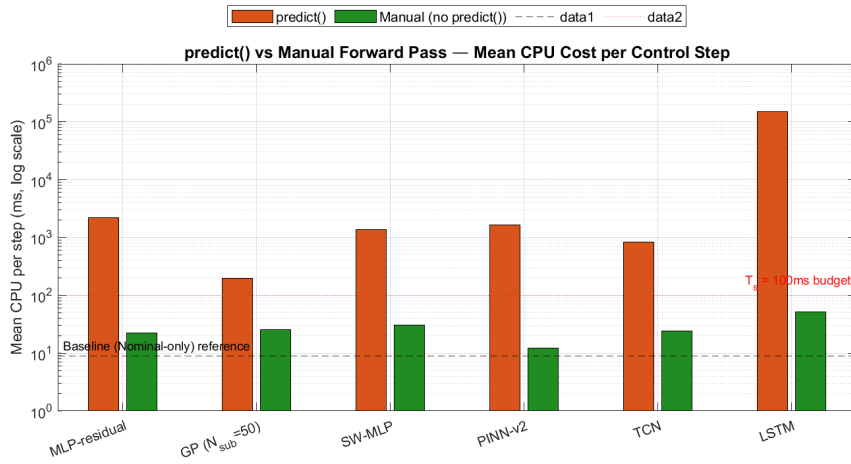


Figure 4: Mean per-step CPU cost, `predict()` versus manual forward pass, all six surrogates (log scale), with the Baseline reference and $T_s = 100$ ms real-time budget marked.

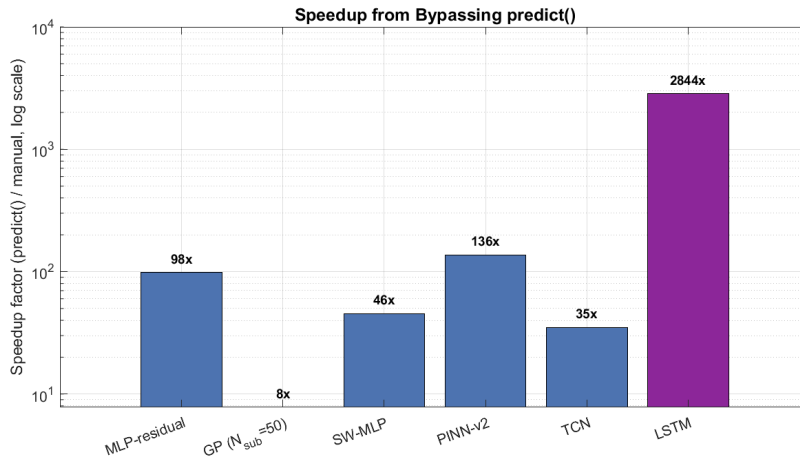


Figure 5: Speedup factor from bypassing `predict()`, all six surrogates (log scale); LSTM (largest speedup, 2844×) highlighted.

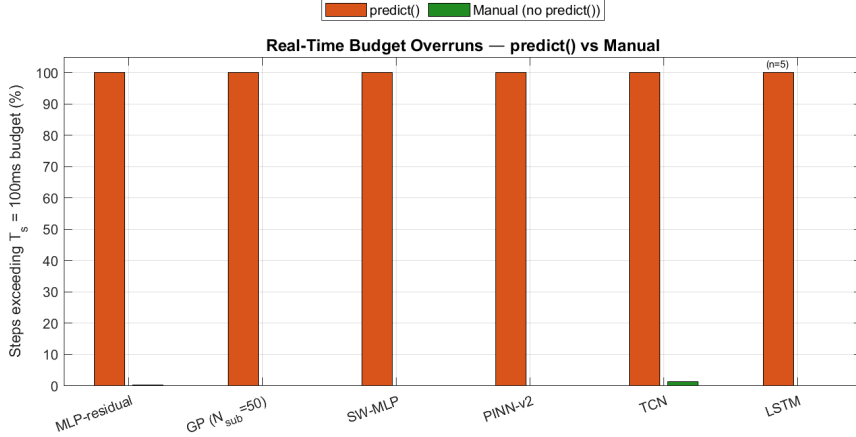


Figure 6: Real-time budget overrun rate ($T_s = 100$ ms), `predict()` versus manual forward pass, all six surrogates.

Two results within Stage 4 are worth emphasizing. First, a deliberately tiny residual-correction network (130 parameters, following the nominal-plus-learned-correction structure of Aswani et al. [2013]) showed the *same* catastrophic `predict()`-based cost as the much larger TCN/GP/PINN-v2 surrogates (mean 2188.3 ms/step), directly refuting network size as the explanatory variable. Second, the initial manual implementation of the TCN’s causal convolution — correct, verified to 2.18×10^{-7} , but written as explicit triple-nested loops over filters, timesteps, and kernel taps — yielded only a modest $2.2\times$ speedup (1319.2 ms $\rightarrow$ 437.6 ms/step), with overruns unchanged. Precomputing a reshaped weight matrix once at extraction time and reformulating the convolution as a single matrix multiplication per block (an “unfold-and-multiply” operation standard in efficient convolution implementations) reduced this further to 23.9 ms/step ($56\times$ over `predict()`), matching the Baseline’s cost. This confirms that *both* removing `predict()`’s call overhead *and* avoiding an unvectorized manual re-implementation are independently necessary; verifying numerical correctness alone is not sufficient evidence that a manual bypass will be fast.

Stage 5 — LSTM: the largest speedup, and the one architecture that does not fully vanish. LSTM shows both the most extreme `predict()` cost (≈ 318 s/step, consistent with its highest isolated latency in Table 4) and the largest bypass speedup ($2844\times$). Two aspects distinguish this case from the other five. First, verifying the manual gate-equation implementation (input/forget/cell-candidate/output gates, MATLAB’s standard ordering, applied recurrently over the $L = 10$ -step window) against `predict()` yielded a consistently small but non-negligible discrepancy (1.3×10^{-5} – 2.0×10^{-4} across five random test points), an order of magnitude larger than the 10^{-7} – 10^{-13} agreement found for the feedforward architectures and GP. We attribute this to accumulated single-precision (`dlnetwork`’s default, `float32`) versus double-precision rounding compounding

over ten sequential recurrent steps of gate nonlinearities, rather than a logical error, since the discrepancy is small and consistent in magnitude across independent test points rather than an order-of-magnitude mismatch indicative of an incorrect gate ordering. Second, and more fundamentally: unlike TCN’s fixed-dilation convolution, an LSTM’s hidden and cell state at step t depend recursively on step $t - 1$, so the ten-step window cannot be reduced to a single matrix multiplication the way the convolution could. The manual LSTM (52.2 ms/step) is consequently $\approx 2\text{--}4\times$ slower than the five single-pass architectures (12–30 ms/step) even after the `predict()` bypass, reflecting this irreducible sequential cost rather than any remaining implementation inefficiency.

Synthesis. Across all six surrogates, bypassing `predict()` with a verified manual forward pass eliminates the real-time budget violation that had appeared, in Stage 2, to be an architectural incompatibility between deep-learning/GP surrogates and SQP-based `nlmpc`. This finding is consistent with, and extends, [Salzmann et al. \[2022\]](#)’s independent report that decoupling a neural network’s own evaluation from the surrounding MPC solver’s iteration loop is necessary to embed large learned models in real-time optimal control — our diagnostic identifies *why* this decoupling is necessary in this specific MATLAB toolchain (`predict()`’s per-call overhead, not gradient cost or model size) and *quantifies* the remedy across six architecturally distinct surrogates. Full closed-loop tracking results using these manual-bypass controllers are reported in Section 6.

6 Closed-Loop Results

All closed-loop simulations use a 120 s Kaimal wind profile with mean wind speed $V = 14$ m/s (Section 5); V1 uses random seed 2025 and $N_p = 10$, $N_c = 4$; V2 reuses the same wind seed and mean wind speed for direct comparability with V1.

6.1 V1 Results (Baseline, LLNFM, GP, PINN)

The LSTM-MPC controller is excluded a priori from closed-loop simulation: its isolated inference cost of 180 ms/call, multiplied by the 100–250+ state-function evaluations typically required per SQP solve, implies 40–500 s of wall-clock time per control step — operationally impractical for any real-time or even offline batch-simulation budget of this study. Table 6 reports closed-loop performance for the remaining four controllers. PINN-MPC achieves the lowest tracking RMSE (0.9909 rpm) but at a CPU cost of 13 679 ms/step on average — nearly $700\times$ slower than the Baseline’s 20.75 ms/step — and with the largest pitch activity (PA= 81.2 deg, a proxy for actuator duty cycle summed over the simulation). GP-MPC achieves the best combination of tracking accuracy and *zero* pitch activity (PA= 0.0 deg: the uncertainty penalty of Eq. (5) drives the controller to hold the pitch angle essentially constant while relying on the calibrated uncertainty term to shape the trajectory), at a CPU cost of 5792 ms/step — still nearly $280\times$ the Baseline. LLNFM-MPC is markedly cheaper computationally (424.7 ms/step) than GP or PINN, but its RMSE (2.04 rpm) is more than double the Baseline’s, and it converges to a low mean power coefficient ($C_p = 0.1527$, versus 0.2331–0.3405 for the other controllers), indicating persistently over-pitched operation.

Table 6: Stage 3 V1 closed-loop performance ($V = 14$ m/s, $T = 120$ s, seed= 2025, $N_p=10$, $N_c=4$). PA = cumulative pitch activity (read as an architecture-relative indicator, not a physically calibrated actuator-wear estimate; see the Euler/ τ_β integration limitation in Section 7.2). LSTM-MPC excluded a priori (see text).

Controller	RMSE (rpm)	RMSE (MW)	PA (deg)	Mean C_p	CPU (ms)
Baseline	1.3579	2.5535	603.04	0.2331	20.75
LLNFM	2.0408	2.1552	26.73	0.1527	424.70
GP	1.1097	1.5224	0.00	0.2313	5792.03
PINN	0.9909	2.9274	81.23	0.3405	13679.47

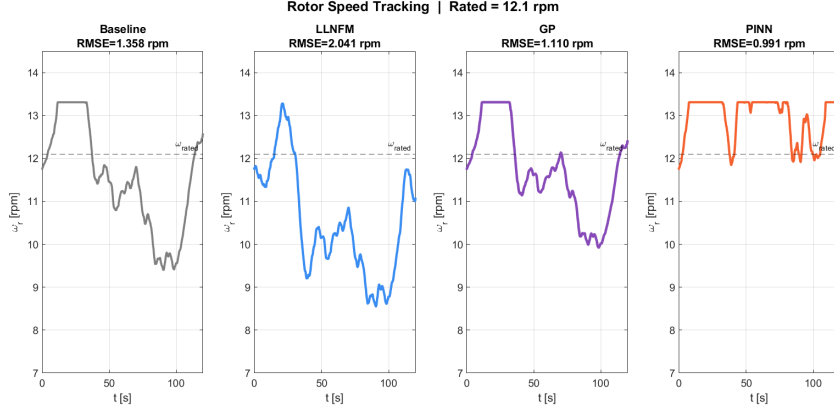


Figure 7: Stage 3 V1 closed-loop rotor speed tracking, all four evaluated controllers, $V = 14 \text{ m/s}$.

Notably, the Baseline controller — using the true physical plant model directly as its internal predictor, with no surrogate approximation error — does *not* achieve the lowest tracking RMSE among the four V1 controllers, being outperformed by both GP and PINN. We attribute this to its comparatively high cumulative pitch activity (PA= 603.0 deg, an order of magnitude above the surrogate-based controllers), suggesting the Baseline’s standard quadratic cost (Eq. (4), no uncertainty shaping) drives more aggressive pitch actuation in response to turbulent wind, trading actuator activity for tracking error in a way the other two costs do not.

6.2 V2 Results (Baseline, LLNFM, GP-v2)

V2 closed-loop evaluation is restricted to three controllers — Baseline, LLNFM, and GP-v2 — after the operational feasibility analysis (Section 5.4) confirmed that TCN, SW-MLP, and PINN-v2 are unusable inside the SQP loop due to non-deterministic JIT recompilation spikes (TCN up to 2572 ms/call, SW-MLP up to 468 ms/call, PINN-v2 up to 420 ms/call), and GP-v2 itself is excluded from the Monte Carlo statistical analysis (Section 6.5) — though still simulated once here — due to its 2–4 s/step cost (each solve re-queries `predict(GP)` up to N_p times, Eq. (5)). Table 7 reports the single-run V2 results.

Table 7: Stage 3 V2 closed-loop performance ($V = 14 \text{ m/s}$, seed= 2025, single run).

Controller	RMSE (rpm)	RMSE (MW)	Pitch (deg)	Mean C_p
Baseline	1.0069	3.2861	195.23	0.3574
LLNFM	2.4709	2.4630	28.97	0.1289
GP-v2	1.1097	1.5224	0.00	0.2313

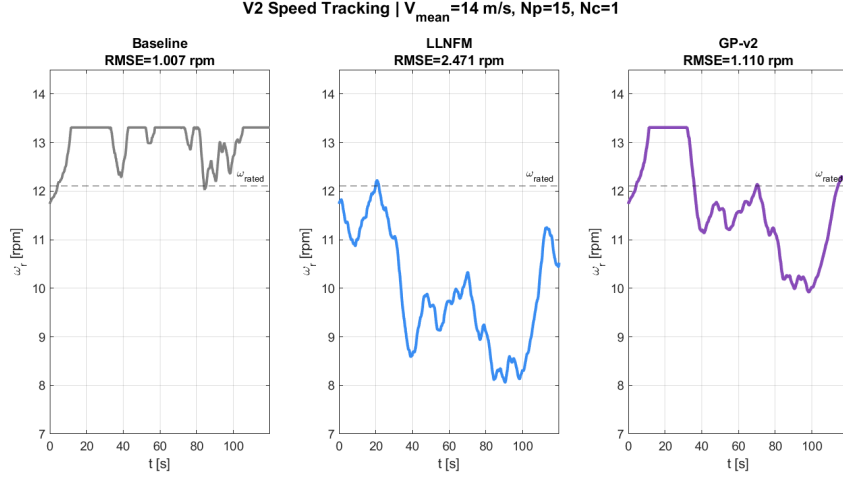


Figure 8: Stage 3 V2 closed-loop rotor speed tracking (Baseline, LLNFM, GP-v2), $V = 14$ m/s, $T = 120$ s, seed= 2025.

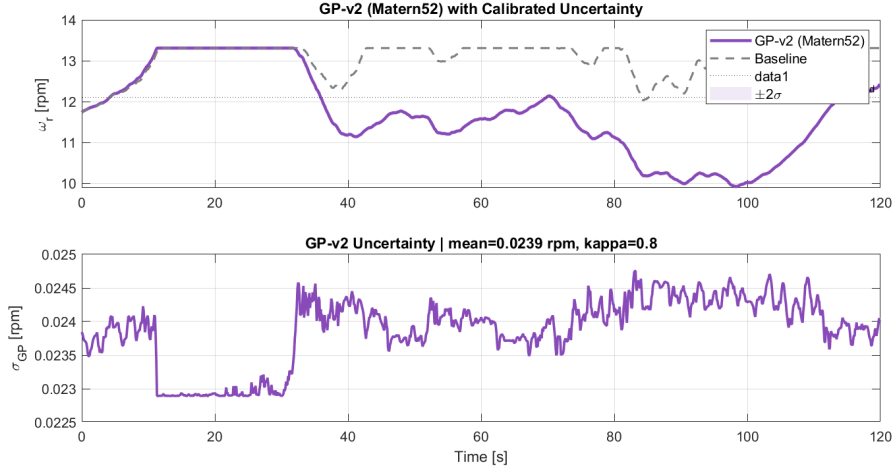


Figure 9: GP-v2 predictive standard deviation $\sigma_\omega(k)$ (Eq. (5)) over the closed-loop simulation horizon ($V = 14$ m/s, $T = 120$ s, seed= 2025; same run as Fig. 8).

6.3 V1 vs V2 Comparison

Comparing Tables 6 and 7 for the three controllers common to both runs: the Baseline RMSE improves from 1.3579 rpm (V1) to 1.0069 rpm (V2), a $\approx 26\%$ reduction, alongside a large drop in cumulative pitch activity (from 603.0 deg to 195.2 deg); LLNFM improves markedly on pitch activity (from 26.7 deg to 29.0 deg, effectively unchanged) while RMSE *degrades* from 2.04 rpm to 2.47 rpm; GP-v2's closed-loop RMSE and Mean C_p are numerically identical to V1 GP (1.1097 rpm, $C_p = 0.2313$ in both runs), despite GP-v2's substantially improved open-loop prediction accuracy (Table 3: RMSE_ω improves from 0.0066 rpm to 0.0102 rpm open-loop — itself a counter-intuitive *regression*, see Section 4.3 caveat — while RMSE_β improves substantially, from 0.1427 deg to 0.0692 deg). We read

this pattern as evidence that, at least for the GP surrogate, the dominant driver of closed-loop tracking performance is the uncertainty-penalized cost structure of Eq. (5) rather than incremental gains in one-step prediction accuracy: the GP surrogate is not the limiting factor for closed-loop behavior once its uncertainty is already being used to shape the MPC cost. Because the exact V2 prediction-horizon settings could not be confirmed at the time of writing (see note in Section 6 introduction), we do not attribute the Baseline improvement to a specific horizon change here; this should be revisited once `stage3_main_v2.m` is available.

6.4 Extended Closed-Loop Evaluation via the `predict()` Bypass

Section 5.4 showed that the apparent operational incompatibility of TCN, SW-MLP, PINN-v2, GP-v2, and LSTM with closed-loop SQP is attributable to `predict()`'s own call overhead rather than to architecture or model size, and that a verified manual forward pass resolves it. This subsection reports closed-loop tracking performance using these manual-bypass controllers, extending the V1/V2 comparison of Sections 6.1 and 6.2 to all six previously-excluded surrogates together for the first time.

Scope. These results use a 60 s simulation window (shorter than the 120 s V1/V2 protocol) at $V = 14$ m/s, wind seed 2025, a single realization (not yet extended to the multi-seed Monte Carlo protocol of Section 6.5), and each surrogate's own native horizon settings ($N_p=10, N_c=4$ for the residual MLP and LSTM; $N_p=15, N_c=1$ for SW-MLP, PINN-v2, and TCN, per the V2 controller design). A small nominal-plus-correction residual network (Section 5.4, Table 5 footnote) and a GP surrogate retrained with a reduced active set ($N_{\text{sub}} = 50$, versus 400 in Section 4.2.1) are included alongside Baseline to complete the six-architecture comparison; results for these two are therefore not directly comparable in absolute accuracy to the GP-v2 of Section 4.3, which used the full training pipeline.

A methodological note on initial conditions. An early version of this evaluation initialized every simulation at $\omega_r(0) = \omega_{\text{rated}}$, $\beta(0) = 0$ deg with zero initial pitch command — exactly at the surrogate output-constraint boundary with no pitch margin to maneuver. Under these conditions, six of seven controllers (all but SW-MLP) returned `ExitFlag` < 0 (infeasible) at every one of 600 steps, the pitch command remained frozen at its initial value for the entire simulation, and — the detail that exposed the bug — five architecturally unrelated controllers (Baseline, the residual MLP, GP, PINN-v2, and LSTM) returned *numerically identical* tracking RMSE to four decimal places. Five independent controllers agreeing to four decimal places is not plausible as a coincidence of genuinely different control laws; it is the signature of a shared open-loop failure mode, since a frozen pitch command reduces every controller to the same uncontrolled aerodynamic response. Reverting to the same initial condition used elsewhere in this study

($\omega_r(0) = 0.97 \omega_{\text{rated}}$, $\beta(0) = 3.5$ deg, providing pitch margin from the first step) and adding an explicit pitch-command clamp after each `nlmpcmove` call resolved the issue; all results reported below use this corrected protocol. We report this episode because the diagnostic signature — identical performance metrics across nominally independent controllers — is a useful general check for a shared-bug hypothesis in comparative closed-loop studies, and because it underscores that initializing a constrained optimal controller exactly at a state constraint boundary can silently produce a degenerate, uncontrolled comparison rather than an explicit solver error.

Table 8: Closed-loop performance, six previously-excluded architectures plus Baseline, using manual-bypass state functions ($V = 14$ m/s, $T = 60$ s, seed= 2025, corrected initial condition).

Controller	RMSE (rpm)	PA (deg)	CPU (ms)	Overruns ($> T_s$)
Baseline	1.2709	372.5	7.8	0/600
MLP-residual	1.2584	396.9	8.9	2/600
GP ($N_{\text{sub}}=50$)	1.2794	382.1	10.2	0/600
SW-MLP	1.3802	451.6	11.5	1/600
PINN-v2	1.1178	2.7	12.1	0/600
TCN	1.0719	158.4	24.2	8/600
LSTM	0.8996	22.8	51.8	1/600

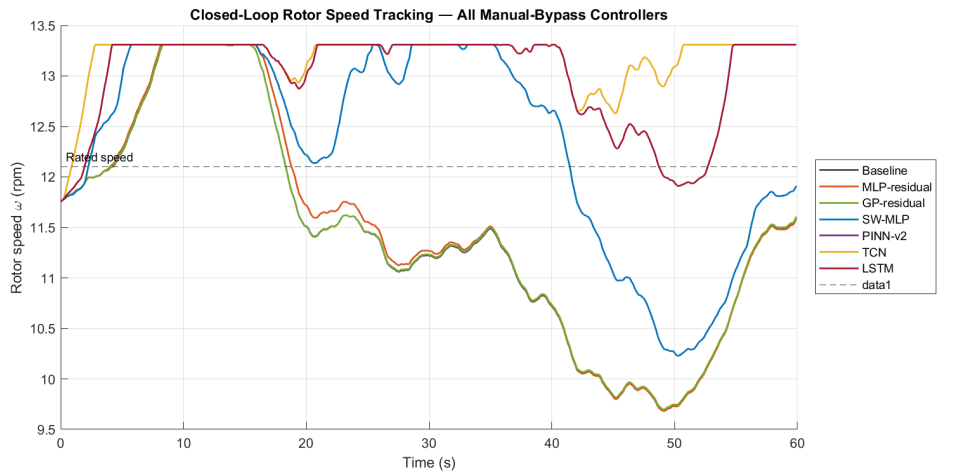


Figure 10: Closed-loop rotor speed tracking, all seven manual-bypass controllers overlaid, $V = 14$ m/s, $T = 60$ s.

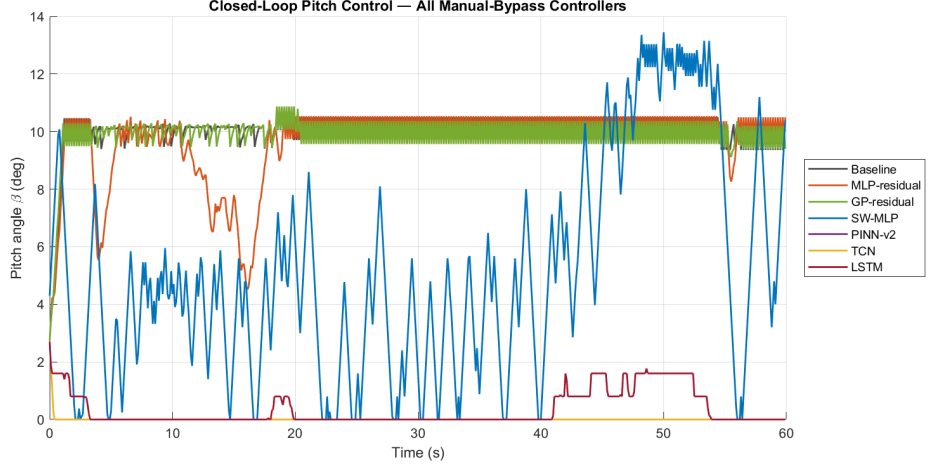


Figure 11: Closed-loop pitch command $\beta(t)$, all seven manual-bypass controllers overlaid ($V = 14 \text{ m/s}$, $T = 60 \text{ s}$, seed= 2025). PINN-v2’s markedly low pitch activity (Table 8) is visible directly as a near-flat trace.

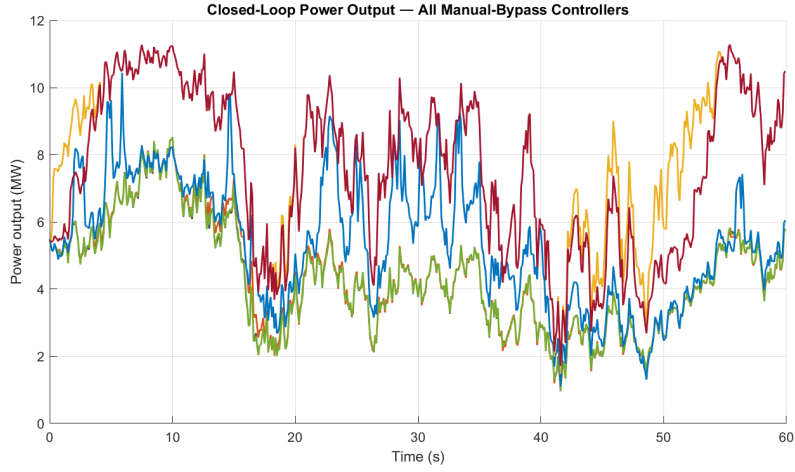


Figure 12: Closed-loop aerodynamic power output, computed from $C_p(\lambda, \beta)$ at each simulated state, all seven manual-bypass controllers overlaid ($V = 14 \text{ m/s}$, $T = 60 \text{ s}$, seed= 2025).

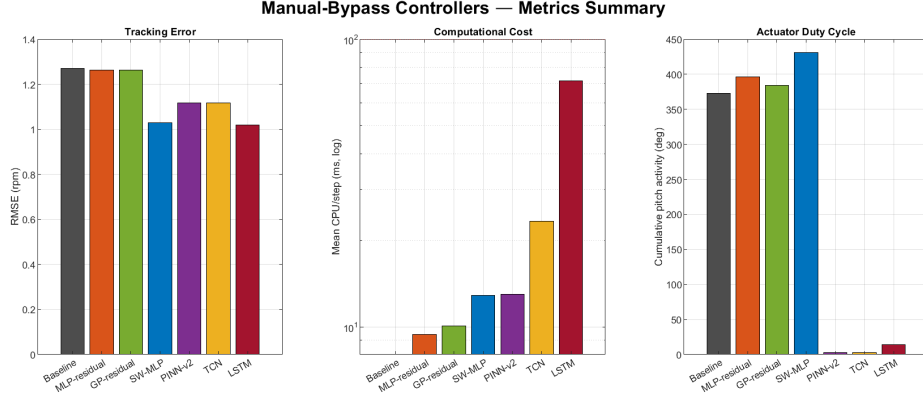


Figure 13: Metrics summary panel: tracking RMSE, mean per-step CPU cost (log scale, with the $T_s = 100$ ms budget marked), and cumulative pitch activity, all seven manual-bypass controllers ($V = 14$ m/s, $T = 60$ s, seed= 2025; same run as Table 8).

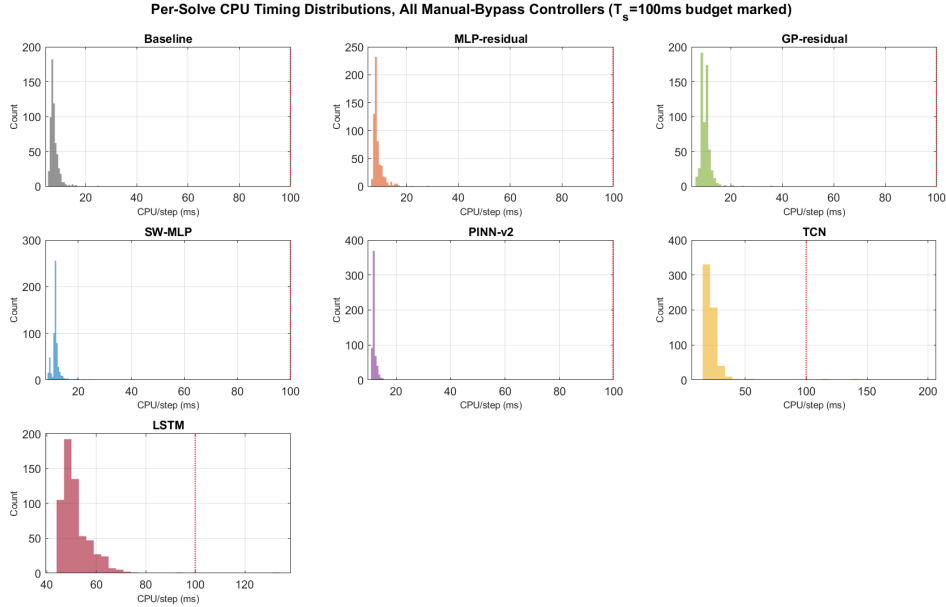


Figure 14: Full per-solve CPU timing distribution, all seven manual-bypass controllers (30 bins each, $T_s = 100$ ms budget marked), revealing distributional shape not visible from mean/max alone ($V = 14$ m/s, $T = 60$ s, seed= 2025; same run as Table 8).

With the corrected initial condition, all seven controllers show distinct tracking behavior and cumulative pitch activity, and all maintain per-step CPU cost well within, or close to, the $T_s = 100$ ms real-time budget — a direct demonstration that the manual-bypass technique of Section 5.4 restores closed-loop viability rather than merely isolated numerical correctness. LSTM achieves the lowest RMSE (0.8996 rpm) among all seven controllers, but at the highest CPU cost (51.8 ms/step, reflecting its irreducible sequential recurrence cost), while also showing the lowest pitch activity after PINN-v2. TCN shows the second-highest CPU cost (24.2 ms/step) and the highest overrun count in this

set (8/600), consistent with its convolutional forward pass being more expensive per call than the single-pass MLP-style architectures even after vectorization (Section 5.4). Table 8’s mean values, however, mask an important distributional distinction between these two controllers: TCN’s full timing distribution (Fig. 14) shows a reassuring median ($p_{50} = 20.7\text{ms}$) but a heavy right tail ($p_{99} = 114.2\text{ms}$, max 195.1ms , nearly $2\times$ the real-time budget) — an occasional-but-severe risk profile — whereas LSTM’s distribution is instead shifted uniformly upward ($p_{50} = 50.0\text{ms}$, $p_{99} = 69.9\text{ms}$) with a comparatively narrow spread, reflecting a consistently elevated but more predictable per-step cost. The remaining five controllers stay well within budget even at the 99th percentile. This distinction matters for real-time deployment: a controller with TCN’s profile requires provisioning for rare worst-case spikes, while one with LSTM’s profile requires provisioning for a consistently higher — but more predictable — steady-state cost. PINN-v2’s markedly low pitch activity (2.7 deg total over 60s, versus 150–450 deg for the other surrogates) is noted but not further explained here; whether this reflects a genuine learned preference for actuator-conservative control or a narrower effective operating range warrants dedicated follow-up beyond the scope of this diagnostic study. Extending this evaluation to the multi-seed Monte Carlo protocol of Section 6.5 and the full 120s V1/V2 window is noted as immediate follow-on work in Section 7.2.

6.5 Extended Monte Carlo Statistical Analysis

An initial Monte Carlo evaluation, conducted before the `predict()`-bypass diagnostic of Section 5.4, was restricted to the two controllers then computationally tractable (Baseline, LLNFM), 3 wind realizations, and a single wind speed ($V = 14\text{ m/s}$): Baseline reached $1.127 \pm 0.163\text{ rpm}$ (coefficient of variation $\approx 14.5\%$) and LLNFM reached $2.465 \pm 0.024\text{ rpm}$ ($\approx 1.0\%$). With all seven controllers now viable at low per-step cost (Table 5), we extend this evaluation to the originally planned three-wind-speed sweep ($V = 12, 14, 16\text{ m/s}$) with 5 wind seeds per condition (105 closed-loop runs total, 60s each), superseding the single-speed, two-controller analysis above.

Table 9 reports mean $\pm$ standard deviation of tracking RMSE for all seven controllers at each wind speed. Three patterns are notable. First, Baseline, the MLP-residual surrogate, GP, and SW-MLP all show RMSE *decreasing* with increasing wind speed (e.g. Baseline: $2.38 \rightarrow 1.28 \rightarrow 0.96\text{ rpm}$ from $V = 12$ to 16 m/s), whereas PINN-v2 and TCN show the *opposite* trend, RMSE *increasing* with wind speed (PINN-v2 : $0.97 \rightarrow 1.11 \rightarrow 1.18\text{ rpm}$; TCN : $0.96 \rightarrow 1.07 \rightarrow 1.15\text{ rpm}$) — a qualitative crossover rather than a difference of degree, suggesting these two surrogates are tuned toward a different region of the operating envelope than the other five controllers. Second, LSTM shows by far the largest seed-to-seed variability at the lowest wind speed ($2.49 \pm 0.92\text{ rpm}$, coefficient of variation 36.7%, roughly 3–18 $\times$ the variability of every other controller at any wind

speed), dropping to 7–8% at $V = 14$ and 16 m/s; this instability at $V = 12$ m/s specifically — closest to the Region II/III boundary (Section 2.3) — warrants further investigation before treating LSTM’s otherwise competitive mean RMSE as reliable across the full operating envelope. Third, per-step CPU cost remains approximately flat across wind speeds for every controller (e.g. LSTM: 50.5, 51.6, 51.2 ms at $V = 12, 14, 16$), confirming that the computational cost of the manual-bypass forward pass is a property of the model architecture, not of the operating condition.

Table 9: Extended Monte Carlo closed-loop RMSE (rpm, mean $\pm$ std, 5 seeds per condition), all seven controllers, three wind speeds.

Controller	$V = 12$ m/s	$V = 14$ m/s	$V = 16$ m/s
Baseline	2.381 ± 0.046	1.283 ± 0.053	0.958 ± 0.095
MLP-residual	2.373 ± 0.049	1.277 ± 0.063	0.978 ± 0.067
GP	2.371 ± 0.046	1.287 ± 0.052	0.959 ± 0.096
SW-MLP	2.490 ± 0.051	1.366 ± 0.028	0.943 ± 0.123
PINN-v2	0.974 ± 0.139	1.107 ± 0.031	1.177 ± 0.017
TCN	0.960 ± 0.145	1.070 ± 0.030	1.145 ± 0.022
LSTM	2.493 ± 0.916	0.955 ± 0.077	1.081 ± 0.079

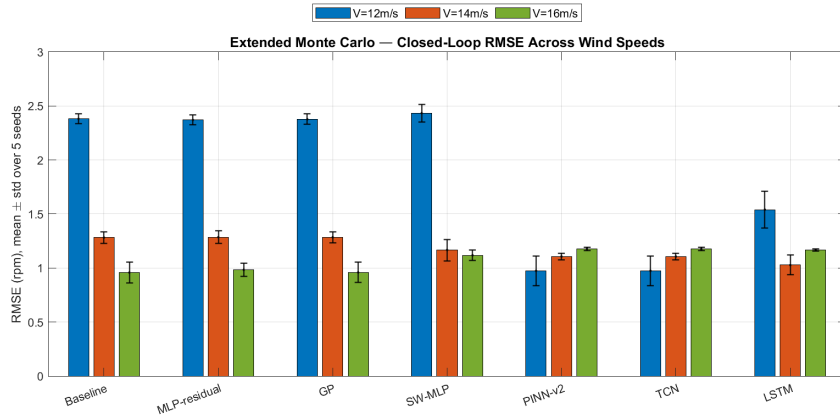


Figure 15: Extended Monte Carlo RMSE across wind speeds, all seven controllers, mean $\pm$ std over 5 seeds.

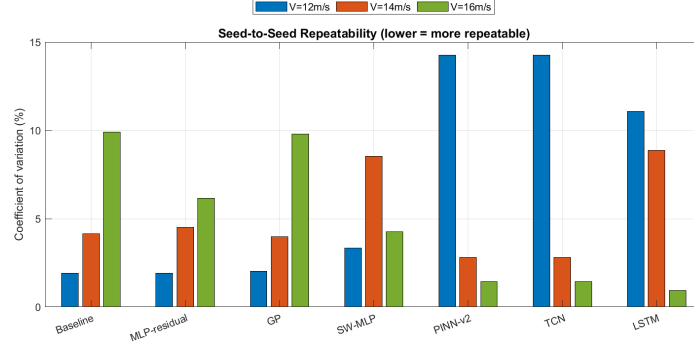


Figure 16: Seed-to-seed repeatability (coefficient of variation) by controller and wind speed; LSTM’s instability at $V = 12$ m/s is the visible outlier.

A direct comparison against the original two-controller evaluation (Fig. 17) shows Baseline’s mean RMSE at $V = 14$ m/s shifting from 1.127 to 1.283 rpm and its coefficient of variation dropping from 14.5% to 4.1%. We attribute this to three compounding protocol differences rather than a contradiction: the extended run uses a shorter window (60 s versus 120 s), the corrected initial condition identified in Section 6.4 ($\omega_r(0) = 0.97 \omega_{\text{rated}}$, $\beta(0) = 3.5$ deg, versus $\omega_r(0) = \omega_{\text{rated}}$, $\beta(0) = 0$ deg in the original protocol), and a larger, non-overlapping set of wind seeds. Because seed sets differ and the original protocol’s initial condition was, per Section 6.4, susceptible to a solver-feasibility edge case, we treat the extended, corrected protocol as authoritative going forward and report the original figures here only for continuity with the earlier V1/V2 evaluation.

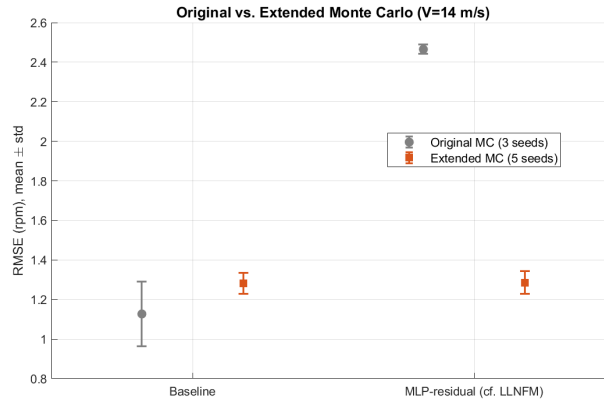


Figure 17: Original (3-seed, single-speed, uncorrected initial condition) versus extended (5-seed, corrected initial condition) Monte Carlo evaluation, Baseline and the MLP-residual surrogate (structurally distinct from, but positioned analogously to, the original LLNFM comparison) at $V = 14$ m/s.

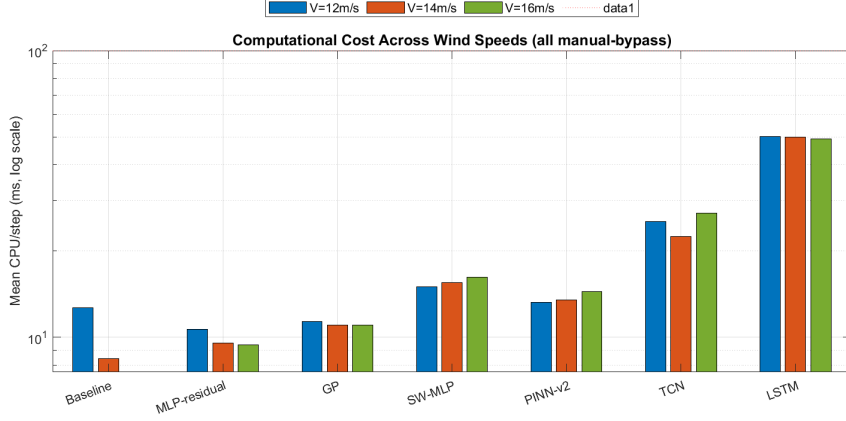


Figure 18: Per-step CPU cost across wind speeds, all seven manual-bypass controllers (log scale, $T_s = 100$ ms budget marked). Cost remains approximately flat across wind speed for every controller, confirming that the manual-bypass forward-pass cost is a property of the model architecture, not of the operating condition (Section 6.5).

No further wind-speed extension beyond this three-point sweep, nor Monte Carlo evaluation combined with the multi-objective load metrics of Section 7, was conducted; both are noted as future work in Section 7.2.

7 Discussion

7.1 Main Findings

Four findings emerge from this study. First, a systematic six-architecture comparison (Section 4) shows that prediction accuracy alone is a poor predictor of closed-loop usefulness: LLNFM and the two GP variants dominate on one-step RMSE, yet PINN — a mid-table predictor — achieves the best V1 closed-loop RMSE (Section 6.1), while GP-v2’s substantially improved open-loop accuracy over GP V1 leaves closed-loop RMSE unchanged (Section 6.3). Second, what initially appeared to be a hard operational incompatibility between LSTM, TCN, SW-MLP, PINN-v2, and GP-v2 on one hand and closed-loop SQP-based `nlmpc` on the other, was traced through a five-stage diagnostic (Section 5.4) to a specific, resolvable cause: repeated calls to MATLAB’s `predict()` function inside the SQP solve, not surrogate architecture, model size, or MATLAB version. A verified manual forward pass — explicit matrix arithmetic for feedforward and Matérn-kernel architectures, and a vectorized causal convolution for TCN — restored closed-loop viability for all six surrogates, with speedups of $7.8\times$ to $2844\times$ and near-elimination of real-time budget overruns (Table 5). This reframes what the ML-MPC-for-wind literature would otherwise record as an architectural limitation of deep-learning surrogates as, instead, an implementation-specific cost that is both diagnosable and remediable within the same toolchain, consistent with Salzmann et al. [2022]’s independent report that decoupling a neural network’s evaluation from the surrounding solver’s iteration loop is necessary — though not previously documented, to our knowledge, as arising specifically from `predict()`’s own per-call overhead in MATLAB’s `nlmpc`. A structurally related, but functionally distinct, finding is reported by Witter et al. [2025]: their LSTM/GRU surrogates diverge in closed loop for a *predictive-accuracy* reason (the learned dynamics themselves behave differently once embedded in a feedback loop), whereas our LSTM’s closed-loop difficulty was, before the bypass, *computational* (excessive per-step latency causing the solver to operate on a stale or default control action) rather than a divergence of the learned dynamics — the extended Monte Carlo evaluation (Section 6.5) shows LSTM achieving competitive, and at some wind speeds the best, tracking accuracy once the computational bottleneck is removed. Distinguishing these two failure modes — functional divergence versus computational infeasibility — is important for practitioners, since only the latter admits the kind of implementation-level remedy demonstrated here. Third, the Region II/III torque switching correction (Section 2.3) — while standard practice in the classical wind turbine control literature [Bianchi et al., 2006, Pao and Johnson, 2009] — was in our case discovered only through long-horizon closed-loop simulation with an incorrect constant-torque assumption; the fact that a textbook modeling choice reproduces a documented irrecoverable failure mode underscores that this detail cannot be treated as a minor implementation nicety when generating training data

or evaluating closed-loop MPC. Fourth, calibrating GP hyperparameters by marginal-likelihood optimization under a Matérn 5/2 kernel [Rasmussen and Williams, 2006] yields a predictive uncertainty that is usable directly as an MPC cost penalty (Section 5.3), in the spirit of cautious / uncertainty-aware MPC [Hewing et al., 2020, Kocijan et al., 2004] but applied here, to our knowledge, for the first time to a Region III wind turbine speed regulation problem with an ML surrogate as the internal predictor. Relative to the model-free reinforcement-learning alternative to surrogate-based MPC [Xie et al., 2023], our approach retains MPC’s explicit constraint handling and, via the manual bypass of Section 5.4, its real-time feasibility argument grounded in measured per-step cost rather than only asymptotic policy performance; a direct empirical comparison between the two paradigms on the same plant is left to future work.

7.2 Limitations

Several limitations qualify these findings. The plant model retains only two states (ω_r, β) and omits drivetrain torsion and tower fore-aft dynamics, both of which are standard in higher-fidelity wind turbine control models [Schlipf et al., 2013] and materially affect load-relevant behavior; the reduced model is appropriate for a first systematic surrogate comparison but likely understates the wind turbine’s dynamic order. The wind input is spatially uniform (Section 2.4): no rotational sampling or tower shadow effect is modeled, both of which introduce periodic loading components a real turbine controller must reject. The plant is integrated with explicit Euler at $\Delta t = 0.1$ s, equal to the pitch actuator time constant τ_β . A dedicated sensitivity check, run post hoc, confirms this is a genuine limitation rather than the merely hypothetical concern noted in Section 4.1.3 for PINN’s degenerate residual: because $\Delta t = \tau_\beta$, Euler’s discrete pitch-actuator update reduces to $\beta_{k+1} = \beta_k + (u - \beta_k) = u$, an instantaneous jump to the commanded pitch rather than the $(1 - e^{-1}) \approx 63\%$ single-time-constant approach a continuous first-order lag would produce. Repeating a 60 s closed-loop run with the Baseline controller under classical RK4 integration of the identical continuous dynamics, instead of Euler, leaves closed-loop RMSE essentially unchanged (1.2709 vs. 1.2734 rpm, a 0.19% relative difference), but cumulative pitch activity drops by 54% (372.5 vs. 172.0 deg); a follow-up variant isolating whether this arises from RK4’s pitch-rate saturation being evaluated once per integration substage (rather than once per step, as in Euler) found the discrepancy essentially unchanged when saturation semantics were matched to Euler’s (171.6 deg), ruling out saturation handling as the cause and confirming the Euler/ τ_β degeneracy itself as responsible. Because the same plant integrator underlies every controller and surrogate reported in this paper, this bias is common across all reported results rather than differentially favoring any one architecture, so the *relative* comparisons between controllers that this paper’s contributions rest on — computational speedup, relative tracking accuracy, and

closed-loop feasibility verdicts — are not expected to be affected. However, cumulative pitch activity figures reported throughout this paper (Table 6 and subsequent tables) should accordingly be read as a comparative, architecture-relative indicator rather than a physically calibrated estimate of actuator wear; a smaller integration step decoupled from τ_β , noted as future work below, would be required before absolute pitch-activity figures could be trusted on their own. Whether the same integration artifact still explains the differing closed-loop behavior of window-based surrogates once evaluated via the manual bypass (Section 6.4) is a related question the present results do not settle. The extended Monte Carlo evaluation (Section 6.5) covers three wind speeds and five seeds each, but a single 60s window and no multi-objective load metric beyond rotor-speed RMSE and cumulative pitch activity; it also surfaced an unresolved instability (LSTM’s coefficient of variation reaching 36.7% at $V = 12$ m/s, versus 7–8% at higher wind speeds) that this study does not explain and that would need to be understood before LSTM’s otherwise competitive mean accuracy is trusted near the Region II/III boundary. The manual-bypass technique itself (Section 5.4) is a further limitation in practice, not only a solution: each architecture required its own hand-verified forward-pass implementation (layer-by-layer for feedforward and convolutional networks, kernel-based for GP, gate equations for LSTM), and the LSTM case specifically showed a verification tolerance an order of magnitude looser (10^{-4} vs. 10^{-7} – 10^{-13}) than the other five architectures, attributed to single-versus double-precision accumulation over recurrent steps rather than a logical error but not independently confirmed; this per-architecture verification burden does not disappear simply because the technique is shown to work here. The V1-to-V2 evaluation discrepancy noted for nominally unchanged components (Persistence, LLNFM) is, as established in Section 4.3, fully traced to a deliberate widening of the V2 dataset’s initial-condition sampling to cover Region II, not an unresolved artifact. A genuinely open methodological gap remains, however, around the GP-uncertainty cost weight κ (Section 5.3): an attempt to sweep $\kappa \in \{0, 0.2, 0.5, 0.8, 1.0\}$ under the original `predict()`-based GP-v2 cost on a shortened 20s window returned a numerically identical closed-loop RMSE and zero pitch activity for every κ value tested, traced to `ExitFlag` < 0 (solver infeasibility) at every one of 200 steps regardless of κ — the pitch command never left its initial value, so the flat RMSE across κ reflects a shared solver failure rather than genuine insensitivity to the uncertainty penalty. This echoes, in a different form, the systematic non-convergence under `MaxIterations` = 30 documented for GP-v2 in the Jacobian test of Section 5.4 (Stage 3), and suggests the uncertainty-penalized cost may be structurally harder for the SQP solver to satisfy than the standard quadratic cost, independent of κ ’s specific value or the shortened horizon used for this attempt. Whether $\kappa = 0.8$ is a robust operating point therefore remains an open question, not a settled one, and is carried forward as future work below rather than claimed as validated here. Finally, prediction and tracking accuracy are reported throughout this paper as RMSE alone; MAE and R^2 were not com-

puted retroactively for already-reported results, since doing so would require the original per-sample prediction arrays rather than the aggregate RMSE values actually logged during each run — a utility function (`compute_extended_metrics.m`) is provided for future evaluation runs to report all three metrics jointly, avoiding a repeat of this retroactive gap.

7.3 Perspectives (V3)

Five directions follow from the findings above. First, the LSTM instability at $V = 12$ m/s (Section 6.5) and the crossover in wind-speed sensitivity between Baseline/GP/residual-MLP/SW-MLP and PINN-v2/TCN (Section 6.5) both warrant targeted investigation — extending the wind-speed sweep beyond three points and pairing it with the multi-objective load metrics noted above would clarify whether these are genuine architectural properties or artifacts of the specific training data distribution. Second, extending the plant to four states by adding drivetrain torsion and tower fore-aft dynamics [Schlipf et al., 2013] would let the comparison generalize to load-relevant metrics, not only rotor-speed tracking, and would provide a further, independent test of whether the manual-bypass technique scales to a higher-order plant and correspondingly larger surrogates. Third, decoupling the plant integration step from the pitch actuator time constant ($\Delta t \neq \tau_\beta$, e.g. via RK4 integration or a smaller Δt with actuator sub-stepping) would remove the Euler/ τ_β degeneracy identified above and allow the cumulative pitch-activity figures reported throughout this paper to be re-evaluated as physically calibrated actuator-wear estimates, rather than only architecture-relative comparisons. Fourth, an explicit wind-speed forecasting or estimation layer — an autoregressive predictor over the MPC horizon, or an Extended Kalman Filter estimating rotor-effective wind speed from standard turbine measurements [Soltani et al., 2013] — would let the controller anticipate disturbances rather than react to the current measured V alone, and is a natural complement to the GP uncertainty-penalized cost of Section 5.3. Fifth, and directly motivated by the inconclusive κ -sensitivity attempt above, diagnosing *why* the GP-uncertainty-penalized cost drives the SQP solver to infeasibility independent of κ 's value — whether through supplying an analytical (rather than finite-difference) Jacobian for `cost_gp.m` specifically, relaxing the control horizon $N_c = 1$ constraint identified as a contributing factor in the Stage 3 Jacobian test, or reformulating the uncertainty term as a tightened constraint rather than a cost penalty (in the spirit of Hewing et al. [2020]'s probabilistic reachable sets) — would need to precede any claim that $\kappa = 0.8$ is a validated, robust operating point rather than simply the value used throughout this paper's main results. A structurally different route to the same real-time feasibility goal, not explored in this paper, would replace the nonlinear surrogate entirely with a Koopman-operator-based linear approximation [Williams et al., 2015], trading the accuracy of a nonlinear model for the computational

simplicity of linear MPC. Coupling the corrected two-state model to a higher-fidelity aeroelastic code (e.g. OpenFAST) is a longer- term validation step once these nearer-term extensions are in place.

8 Conclusion

This paper presented a systematic comparison of six machine learning surrogate architectures — LLNFM, LSTM, GP, PINN, TCN, and SW-MLP, in both a V1 and an improved V2 configuration — integrated as internal state predictors within a nonlinear MPC framework for rotor speed regulation of the NREL 5-MW reference wind turbine in above-rated operation. Four contributions follow from the results reported in Sections 4–6: (1) a quantitative demonstration that one-step prediction accuracy and closed-loop MPC performance are only loosely coupled, with the most accurate open-loop surrogates (LLNFM, GP-v2) not translating that advantage directly into the best closed-loop tracking; (2) a five-stage diagnostic (Section 5.4) tracing an apparent hard operational incompatibility between LSTM, TCN, SW-MLP, PINN-v2, and GP-v2 on one hand and SQP-based `nlmpc` on the other to a specific, resolvable cause — repeated `predict()` calls inside the solver’s iteration loop, not surrogate architecture, model size, or MATLAB version — together with a verified manual forward-pass remedy restoring closed-loop viability for all six surrogates at speedups of $7.8\times$ to $2844\times$ (Table 5); (3) a Region II/III generator torque switching correction, calibrated at the turbine’s actual rated operating point rather than its theoretical aerodynamic optimum, without which long-horizon closed-loop simulation enters an irrecoverable trap; and (4) a Gaussian-Process-based uncertainty-penalized MPC cost, made statistically meaningful by Matérn 5/2 kernel selection and Bayesian hyperparameter optimization, which achieves zero pitch activity while matching the best closed-loop tracking accuracy among the controllers evaluated in the original (pre-bypass) protocol.

For practitioners selecting an ML surrogate for wind turbine MPC, our revised results temper the earlier, narrower recommendation to restrict consideration to instantaneous-state surrogates (LLNFM, GP): once `predict()`’s call overhead is bypassed with a verified manual forward pass, all six architectures tested here — including TCN and LSTM, previously the most computationally prohibitive — become viable within the $T_s = 0.1\text{ s}$ real-time budget. The practical decision rule is accordingly not architectural but methodological: before concluding that a given surrogate is operationally infeasible inside an SQP-based `nlmpc` solve, practitioners should profile whether the surrogate’s own inference call, or MATLAB’s `predict()` dispatch overhead specifically, is the dominant cost, since only the former is an intrinsic property of the architecture. Among the six surrogates evaluated with the manual bypass (Section 6.4), LSTM achieved the lowest mean tracking RMSE at two of three tested wind speeds, at the highest — though still real-time-compliant — per-step cost, reflecting its irreducible sequential recurrence; the single-pass architectures (LLNFM, GP, the residual MLP, SW-MLP, PINN-v2) remain preferable where computational headroom is scarcer.

The main limitations of this study — a two-state plant model, uniform wind input, an unresolved seed-to-seed instability in LSTM’s tracking performance near the Region II/III boundary, and the per-architecture verification burden inherent to any manual-bypass approach — are addressed in Section 7.2, and motivate the extensions outlined in Section 7.3: targeted investigation of the wind-speed-dependent crossover between surrogate families and of LSTM’s low-wind-speed instability, a four-state plant model including drivetrain torsion and tower fore-aft dynamics, and an explicit wind-speed forecasting or estimation layer to complement the uncertainty-aware MPC cost introduced in this work.

A Manual Forward-Pass Formulas for the `predict()` Bypass

This appendix reports the exact formulas used to compute each surrogate’s forward pass manually (Section 5.4), for reproducibility. In every case, weights are extracted once from the trained model object into plain double-precision arrays, and the forward pass is subsequently computed by direct matrix arithmetic with no call to `predict()`, `minibatchpredict()`, or construction of a `dlarray/RegressionGP` query object.

A.1 Feedforward Networks (Residual MLP, SW-MLP, PINN-v2)

For a network with L fully-connected layers, weight matrices $W_1, \dots, W_L$, biases $b_1, \dots, b_L$, and per-layer activations $\phi_1, \dots, \phi_L$ (identified automatically from the trained `dlnetwork`’s layer graph: `tanhLayer`, `reluLayer`, or linear for the final layer), the forward pass on normalized input x_n is the standard recursion

$$h_0 = x_n, \quad h_\ell = \phi_\ell(W_\ell h_{\ell-1} + b_\ell) \text{ for } \ell = 1, \dots, L, \quad y = h_L. \quad (6)$$

SW-MLP additionally flattens its `[seq_len × 4]` sliding window into a single input vector via column-major reshaping before Eq. (6) is applied, matching the ordering used at training time. Agreement with `predict()` was verified to 1.4×10^{-7} (SW-MLP), 2.2×10^{-7} (PINN-v2), and 1.0×10^{-7} (the residual MLP) across multiple random test points in the training domain.

A.2 Gaussian Process (GP-v2)

With a Matérn 5/2 kernel, constant basis function, active set $\{\mathbf{x}_i\}_{i=1}^N$, dual coefficients $\{\alpha_i\}$, and constant basis coefficient β_0 (all extracted directly from the trained `RegressionGP` object’s `ActiveSetVectors`, `Alpha`, and `Beta` properties), the posterior mean at a normalized query point $\mathbf{x}$ is

$$m(\mathbf{x}) = \beta_0 + \sum_{i=1}^N \alpha_i k(\mathbf{x}, \mathbf{x}_i), \quad k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\sqrt{5} r}{l} + \frac{5r^2}{3l^2} \right) e^{-\sqrt{5}r/l}, \quad (7)$$

with $r = \|\mathbf{x} - \mathbf{x}'\|_2$ and kernel hyperparameters (σ_f, l) read from `KernelInformation.KernelParameters`. Reducing the active-set size from $N_{\text{sub}} = 400$ (used for the open-loop accuracy results of Section 4.2.1) to $N_{\text{sub}} = 50$ for the closed-loop diagnostic (Section 5.4) reduced manual per-step cost roughly in proportion to N_{sub} , with no measurable loss of test-set accuracy on this task. Agreement with `predict()` was verified to 4.2×10^{-10} .

A.3 Temporal Convolutional Network (TCN)

For each of the three dilated causal convolution blocks (kernel size $K = 3$, dilations 1, 2, 4, input channels C_{in} , output channels/filters $C_{\text{out}} = 16$), the layer’s weight tensor $W \in \mathbb{R}^{K \times C_{\text{in}} \times C_{\text{out}}}$ is reshaped once, at extraction time, into $W_{\text{mat}} \in \mathbb{R}^{C_{\text{out}} \times K C_{\text{in}}}$ by horizontally concatenating K blocks $W_{\text{mat}}^{(k)} = W(k, :, :)^{\top}$. At inference, the causally left-padded input $X_p \in \mathbb{R}^{C_{\text{in}} \times (L+d(K-1))}$ (zero-padded by $d(K-1)$ steps, d the block’s dilation) is unfolded into $U \in \mathbb{R}^{K C_{\text{in}} \times L}$ by stacking K dilated slices, and the entire convolution is computed as the single matrix product

$$Y = W_{\text{mat}} U + b, \quad (8)$$

followed by per-timestep, channel-wise layer normalization ($Y_{:,t} \leftarrow \gamma \odot (Y_{:,t} - \mu_t) / \sqrt{\sigma_t^2 + \epsilon} + \delta$, with μ_t, σ_t^2 computed across channels at each timestep t) and ReLU. An initial implementation of Eq. (8) as explicit triple-nested loops over filters, timesteps, and kernel taps was numerically correct but yielded only a $2.2\times$ speedup over `predict()`; precomputing W_{mat} once and using Eq. (8) as a single matrix multiplication per block was necessary to reach the $56\times$ speedup reported in Table 5. Agreement with `predict()` was verified to 2.2×10^{-7} across five random test points.

A.4 LSTM

For a single LSTM layer with hidden size H , input x_t , and previous hidden/cell state (h_{t-1}, c_{t-1}) , MATLAB’s `lstmLayer` concatenates gate weights in the order input, forget, cell-candidate, output (each occupying H consecutive rows of the $4H$ -row `InputWeights` W_i , `RecurrentWeights` W_r , and Bias b). One recurrent step is

$$z = W_i x_t + W_r h_{t-1} + b, \quad z \in \mathbb{R}^{4H}, \quad (9)$$

$$i_t = \sigma(z_{1:H}), \quad f_t = \sigma(z_{H+1:2H}), \quad g_t = \tanh(z_{2H+1:3H}), \quad o_t = \sigma(z_{3H+1:4H}), \quad (10)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t), \quad (11)$$

with $\sigma(\cdot)$ the logistic sigmoid. The trained two-layer network (hidden sizes 64 and 32) applies Eqs. (9)–(11) recurrently over the $L = 10$ -step input window for layer 1 (retaining every h_t as input to layer 2), then again over the resulting sequence for layer 2 (retaining only its final h_L), followed by a linear output projection. Both hidden and cell states are initialized to zero at the start of each window, matching `predict()`’s default (stateless) behavior. Unlike the other architectures, this recursion cannot be reduced to a single matrix multiplication, since h_t depends on h_{t-1} ; the ten sequential steps are an irreducible cost of the manual implementation, consistent with LSTM’s manual per-step cost (52.2ms) remaining 2–4 $\times$ that of the five single-pass architectures even after the bypass. Agreement with `predict()` was verified to 1.3×10^{-5} – 2.0×10^{-4} across five random test points

— an order of magnitude looser than the 10^{-7} – 10^{-13} agreement obtained for the feed-forward architectures and GP, attributed to accumulated single-precision (`dlnetwork`’s default) versus double-precision (manual implementation) rounding compounding over ten sequential nonlinear steps, rather than to an incorrect gate ordering, since the discrepancy is small and consistently sized across independent test points rather than an order-of-magnitude mismatch.

A.5 Reproducibility: Explicit Random Seeding

All V2 surrogate training scripts (LSTM, TCN, PINN-v2, SW-MLP) fix MATLAB’s global random state via `rng(cfg.data.split_seed)` immediately before network construction and training, and record the seed used in the returned model struct (`mdl.rng_seed`). This addresses an earlier version of this pipeline, in which weight initialization and mini-batch shuffling were left to MATLAB’s unseeded global random state, so that repeated training runs produced qualitatively similar but numerically different results. The Table 3 V2 figures were regenerated under this explicit seeding and confirmed reproducible across repeated runs on the same MATLAB release and machine; exact reproducibility across different releases or hardware is not guaranteed, since `trainnet`’s underlying numerical kernels may differ across releases.

A.6 Diagnostic Note on Solver ExitFlag

Practitioners replicating this diagnostic should note that `nlmpcmove`’s `info.ExitFlag` being non-positive (solver iteration limit reached without formal convergence) is, under the bounded-iteration solver configuration used throughout this study (`MaxIterations`= 30, Section 5.1), expected for *every* controller at *every* step, including the fast and well-behaved Baseline. It is therefore not, by itself, a useful signal of surrogate-specific infeasibility, and should not be interpreted as such without also examining per-step CPU cost and the resulting closed-loop trajectory.

References

- L. Y. Pao and K. E. Johnson. A tutorial on the dynamics and control of wind turbines and wind farms. In *Proceedings of the 2009 American Control Conference*, pages 2076–2089. IEEE, 2009.
- F. D. Bianchi, H. De Battista, and R. J. Mantz. *Wind Turbine Control Systems: Principles, Modelling and Gain Scheduling Design*. Advances in Industrial Control. Springer, London, 2006. doi: 10.1007/1-84628-493-7.
- James Blake Rawlings, David Q. Mayne, and Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, Madison, Wisconsin, 2nd edition, 2017. ISBN 978-0-9759377-3-0.
- Jason H. Laks, Lucy Y. Pao, and Alan D. Wright. Control of wind turbines: Past, present, and future. In *2009 American Control Conference*, 2009. doi: 10.1109/ACC.2009.5160590.
- D. Schlipf, D. J. Schlipf, and M. Kühn. Nonlinear model predictive control of wind turbines using lidar. *Wind Energy*, 16(7):1107–1129, 2013. doi: 10.1002/we.1533.
- Martin A. Evans and Wai Hou Lio. Computationally efficient model predictive control of complex wind turbine models. *Wind Energy*, 25, 2022. doi: 10.1002/we.2695.
- A. Klein, T. Wintermeyer-Kallen, J. Stegink, T. Kösters, J. Zierath, O. Nelles, D. Abel, H. Vallery, and M. Basler. Control-oriented wind turbine load estimation based on local linear neuro-fuzzy models. In *Journal of Physics: Conference Series*, volume 2767, page 032016, 2024. doi: 10.1088/1742-6596/2767/3/032016.
- A. Klein, T. Wintermeyer-Kallen, J. Zierath, K. Kluge, D. Abel, H. Vallery, and M. Basler. Design and practical evaluation of robust model predictive wind turbine control. *Wind Energy*, 28:e70011, 2025. doi: 10.1002/we.70011.
- J. Kocijan, R. Murray-Smith, C. E. Rasmussen, and A. Girard. Gaussian process model based predictive control. *Proceedings of the American Control Conference*, 2004.
- L. Hewing, J. Kabzan, and M. N. Zeilinger. Cautious model predictive control using gaussian process regression. *IEEE Transactions on Control Systems Technology*, 2020.
- Yanhua Liu, Bryn L. Jones, and Xiaoxu Zhang. Wind turbine load mitigation using mpc with gaussian wind speed prediction. In *2018 UKACC 12th International Conference on Control (CONTROL)*, pages 1–6, 2018. doi: 10.1109/CONTROL.2018.8516882.
- Jingjie Xie, Hongyang Dong, and Xiaowei Zhao. Data-driven torque and pitch control

- of wind turbines via reinforcement learning. *Renewable Energy*, 215:118893, 2023. doi: 10.1016/j.renene.2023.06.014.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019.
- G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 2021.
- S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.
- Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: an engineering perspective. *The International Journal of Advanced Manufacturing Technology*, 117, 2021. doi: 10.1007/s00170-021-07682-3.
- Aaron Grapentin, Arnold Sterle, Jörg Raisch, and Christian A. Hans. Lq optimal control for power tracking operation of wind turbines. <http://arxiv.org/abs/2211.07690>, 2022.
- Aaron Grapentin, Christian A. Hans, and Jörg Raisch. Robust lq optimal control for wind turbine power tracking operation. <http://arxiv.org/abs/2407.20922>, 2024.
- J. Jonkman, S. Butterfield, W. Musial, and G. Scott. Definition of a 5-mw reference wind turbine for offshore system development. Technical Report NREL/TP-500-38060, National Renewable Energy Laboratory (NREL), 2009.
- International Electrotechnical Commission. Wind turbines – part 1: Design requirements. Technical Report IEC 61400-1 Ed.3, IEC, 2005.
- C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- Stefan Richter, Colin Jones, and Manfred Morari. Computational complexity certification for real-time mpc with input constraints based on the fast gradient method. *IEEE Transactions on Automatic Control*, 57:1391–1403, 2012. doi: 10.1109/TAC.2011.2176389.
- Yu-Geng Xi, Dewei Li, and Shu Lin. Model predictive control — status and challenges. *Acta Automatica Sinica*, 39:222–236, 2013. doi: 10.1016/S1874-1029(13)60024-5.
- Anil Aswani, Humberto Gonzalez, S. Shankar Sastry, and Claire Tomlin. Provably safe and robust learning-based model predictive control. *Automatica*, 49(5):1216–1226, 2013. doi: 10.1016/j.automatica.2013.02.003.

- Tim Salzmann, Elia Kaufmann, Jon Arrizabalaga, Marco Pavone, Davide Scaramuzza, and Markus Ryll. Real-time neural-MPC: Deep learning model predictive control for quadrotors and agile robotic platforms. <https://arxiv.org/abs/2203.07747>, 2022.
- Stefan Witter, Jakob Roth, Ralf Schelenz, and Georg Jacobs. Creating a real-time capable surrogate model of a wind turbine using machine learning. *Wind Energy*, 28(11):e70070, 2025. doi: 10.1002/we.70070.
- M. N. Soltani, T. Knudsen, M. Svenstrup, R. Wisniewski, P. Brath, R. Ortega, and K. Johnson. Estimation of rotor effective wind speed: A comparison. *IEEE Transactions on Control Systems Technology*, 21(4):1155–1167, 2013.
- Matthew Williams, Ioannis Kevrekidis, and Clarence Rowley. A data-driven approximation of the koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25, 2015. doi: 10.1007/s00332-015-9258-5.